```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.preprocessing import StandardScaler

# Setting untuk membuat angka mudah dibaca di display
pd.options.display.float_format = '{:20.2f}'.format

# Menampilkan semua kolom pada output
pd.set_option('display.max_columns', None)
```

```
df = pd.read_excel("../data/online_retail_II.xlsx", sheet_name=0) # sheet name buat ngebaca bo

df.head(10)
```

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
0	489434	85048	15CM CHRISTMAS GLASS BALL 20 LIGHTS	12	2009-12-01 07:45:00	6.95	13085.00	United Kingdom
1	489434	79323P	PINK CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13085.00	United Kingdom
2	489434	79323W	WHITE CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13085.00	United Kingdom
3	489434	22041	RECORD FRAME 7" SINGLE SIZE	48	2009-12-01 07:45:00	2.10	13085.00	United Kingdom
4	489434	21232	STRAWBERRY CERAMIC TRINKET BOX	24	2009-12-01 07:45:00	1.25	13085.00	United Kingdom
5	489434	22064	PINK DOUGHNUT TRINKET POT	24	2009-12-01 07:45:00	1.65	13085.00	United Kingdom
6	489434	21871	SAVE THE PLANET MUG	24	2009-12-01 07:45:00	1.25	13085.00	United Kingdom
7	489434	21523	FANCY FONT HOME SWEET HOME DOORMAT	10	2009-12-01 07:45:00	5.95	13085.00	United Kingdom
8	489435	22350	CAT BOWL	12	2009-12-01 07:46:00	2.55	13085.00	United Kingdom
9	489435	22349	DOG BOWL , CHASING BALL DESIGN	12	2009-12-01 07:46:00	3.75	13085.00	United Kingdom

```python
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 525461 entries, 0 to 525460
Data columns (total 8 columns):
 #   Column       Non-Null Count   Dtype         
---  ------       --------------   -----         
 0   Invoice      525461 non-null  object        
 1   StockCode    525461 non-null  object        
 2   Description  522533 non-null  object        
 3   Quantity     525461 non-null  int64         
 4   InvoiceDate  525461 non-null  datetime64[ns]
 5   Price        525461 non-null  float64       
 6   Customer ID  417534 non-null  float64       
 7   Country      525461 non-null  object        
dtypes: datetime64[ns](1), float64(2), int64(1), object(4)
memory usage: 32.1+ MB
```

```python
df.describe()
```

	Quantity	InvoiceDate	Price	Customer ID
count	525461.00	525461	525461.00	417534.00
mean	10.34	2010-06-28 11:37:36.845017856	4.69	15360.65
min	-9600.00	2009-12-01 07:45:00	-53594.36	12346.00
25%	1.00	2010-03-21 12:20:00	1.25	13983.00
50%	3.00	2010-07-06 09:51:00	2.10	15311.00
75%	10.00	2010-10-15 12:45:00	4.21	16799.00
max	19152.00	2010-12-09 20:01:00	25111.09	18287.00
std	107.42	NaN	146.13	1680.81

```python
df.describe(include='O') # Mendapatkan deskripsi untuk objek data type (string)
```

	Invoice	StockCode	Description	Country
count	525461	525461	522533	525461
unique	28816	4632	4681	40
top	537434	85123A	WHITE HANGING HEART T-LIGHT HOLDER	United Kingdom
freq	675	3516	3549	485852

```python
df[df['Customer ID'].isna()].head(10)
```

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
263	489464	21733	85123a mixed	-96	2009-12-01 10:52:00	0.00	NaN	United Kingdom
283	489463	71477	short	-240	2009-12-01 10:52:00	0.00	NaN	United Kingdom
284	489467	85123A	21733 mixed	-192	2009-12-01 10:53:00	0.00	NaN	United Kingdom
470	489521	21646	NaN	-50	2009-12-01 11:44:00	0.00	NaN	United Kingdom
577	489525	85226C	BLUE PULL BACK RACING CAR	1	2009-12-01 11:49:00	0.55	NaN	United Kingdom
578	489525	85227	SET/6 3D KIT CARDS FOR KIDS	1	2009-12-01 11:49:00	0.85	NaN	United Kingdom
1055	489548	22271	FELTCRAFT DOLL ROSIE	1	2009-12-01 12:32:00	2.95	NaN	United Kingdom
1056	489548	22254	FELT TOADSTOOL LARGE	12	2009-12-01 12:32:00	1.25	NaN	United Kingdom
1057	489548	22273	FELTCRAFT DOLL MOLLY	3	2009-12-01 12:32:00	2.95	NaN	United Kingdom
1058	489548	22195	LARGE HEART MEASURING SPOONS	1	2009-12-01 12:32:00	1.65	NaN	United Kingdom

```python
df[df["Quantity"]< 0].head(10) # Penasaran dengan apa sih nilai negatif pada data ini
```

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
178	C489449	22087	PAPER BUNTING WHITE LACE	-12	2009-12-01 10:33:00	2.95	16321.00	Australia
179	C489449	85206A	CREAM FELT EASTER EGG BASKET	-6	2009-12-01 10:33:00	1.65	16321.00	Australia
180	C489449	21895	POTTING SHED SOW 'N' GROW SET	-4	2009-12-01 10:33:00	4.25	16321.00	Australia
181	C489449	21896	POTTING SHED TWINE	-6	2009-12-01 10:33:00	2.10	16321.00	Australia
182	C489449	22083	PAPER CHAIN KIT RETRO SPOT	-12	2009-12-01 10:33:00	2.95	16321.00	Australia
183	C489449	21871	SAVE THE PLANET MUG	-12	2009-12-01 10:33:00	1.25	16321.00	Australia
184	C489449	84946	ANTIQUE SILVER TEA GLASS ETCHED	-12	2009-12-01 10:33:00	1.25	16321.00	Australia
185	C489449	84970S	HANGING HEART ZINC T-LIGHT HOLDER	-24	2009-12-01 10:33:00	0.85	16321.00	Australia
186	C489449	22090	PAPER BUNTING RETRO SPOTS	-12	2009-12-01 10:33:00	2.95	16321.00	Australia
196	C489459	90200A	PURPLE SWEETHEART BRACELET	-3	2009-12-01 10:44:00	4.25	17592.00	United Kingdom

```
df["Invoice"] = df["Invoice"].astype("str")
df[df["Invoice"].str.match("^\\d{6}$") == False] # Maksud kodingan ini adalah, kita ingin "mat
```

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
178	C489449	22087	PAPER BUNTING WHITE LACE	-12	2009-12-01 10:33:00	2.95	16321.00	Australia
179	C489449	85206A	CREAM FELT EASTER EGG BASKET	-6	2009-12-01 10:33:00	1.65	16321.00	Australia
180	C489449	21895	POTTING SHED SOW 'N' GROW SET	-4	2009-12-01 10:33:00	4.25	16321.00	Australia
181	C489449	21896	POTTING SHED TWINE	-6	2009-12-01 10:33:00	2.10	16321.00	Australia
182	C489449	22083	PAPER CHAIN KIT RETRO SPOT	-12	2009-12-01 10:33:00	2.95	16321.00	Australia
...	...	...	...	...	...	...	...	...
524695	C538123	22956	36 FOIL HEART CAKE CASES	-2	2010-12-09 15:41:00	2.10	12605.00	Germany
524696	C538124	M	Manual	-4	2010-12-09 15:43:00	0.50	15329.00	United Kingdom
524697	C538124	22699	ROSES REGENCY TEACUP AND SAUCER	-1	2010-12-09 15:43:00	2.95	15329.00	United Kingdom
524698	C538124	22423	REGENCY CAKESTAND 3 TIER	-1	2010-12-09 15:43:00	12.75	15329.00	United Kingdom
525282	C538164	35004B	SET OF 3 BLACK FLYING DUCKS	-1	2010-12-09 17:32:00	1.95	14031.00	United Kingdom

10209 rows × 8 columns

```
df["Invoice"].str.replace("[0-9]", "", regex=True).unique() # Maksudnya yaitu "Tolong hapus se
```

```
array(['', 'C', 'A'], dtype=object)
```

```
df[df["Invoice"].str.startswith("A")]
```

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
179403	A506401	B	Adjust bad debt	1	2010-04-29 13:36:00	-53594.36	NaN	United Kingdom
276274	A516228	B	Adjust bad debt	1	2010-07-19 11:24:00	-44031.79	NaN	United Kingdom
403472	A528059	B	Adjust bad debt	1	2010-10-20 12:04:00	-38925.87	NaN	United Kingdom

```
df["StockCode"] = df["StockCode"].astype("str")
df[(df["StockCode"].str.match("^\\d{5}$") == False)] #buang yang Barang Normal (5 angka) aja.
```

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
1	489434	79323P	PINK CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13085.00	United Kingdom
2	489434	79323W	WHITE CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13085.00	United Kingdom
12	489436	48173C	DOOR MAT BLACK FLOCK	10	2009-12-01 09:06:00	5.95	13078.00	United Kingdom
23	489436	35004B	SET OF 3 BLACK FLYING DUCKS	12	2009-12-01 09:06:00	4.65	13078.00	United Kingdom
28	489436	84596F	SMALL MARSHMALLOWS PINK BOWL	8	2009-12-01 09:06:00	1.25	13078.00	United Kingdom
...	...	...	...	...	...	...	...	...
525387	538170	84029E	RED WOOLLY HOTTIE WHITE HEART.	2	2010-12-09 19:32:00	3.75	13969.00	United Kingdom
525388	538170	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	2	2010-12-09 19:32:00	3.75	13969.00	United Kingdom
525389	538170	85232B	SET OF 3 BABUSHKA STACKING TINS	2	2010-12-09 19:32:00	4.95	13969.00	United Kingdom
525435	538171	47591D	PINK FAIRY CAKE CHILDRENS APRON	1	2010-12-09 20:01:00	1.95	17530.00	United Kingdom
525436	538171	47591B	SCOTTIES CHILDRENS APRON	2	2010-12-09 20:01:00	1.65	17530.00	United Kingdom

80112 rows × 8 columns

```
df[(df["StockCode"].str.match("^\\d{5}$") == False) & (df["StockCode"].str.match("^\\d{5}[a-zA
```

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
89	489439	POST	POSTAGE	3	2009-12-01 09:28:00	18.00	12682.00	France
126	489444	POST	POSTAGE	1	2009-12-01 09:55:00	141.00	12636.00	USA
173	489447	POST	POSTAGE	1	2009-12-01 10:10:00	130.00	12362.00	Belgium
625	489526	POST	POSTAGE	6	2009-12-01 11:50:00	18.00	12533.00	Germany
735	C489535	D	Discount	-1	2009-12-01 12:11:00	9.00	15299.00	United Kingdom
...	...	...	...	...	...	...	...	...
524776	538147	M	Manual	1	2010-12-09 16:11:00	15.00	13090.00	United Kingdom
524887	538148	DOT	DOTCOM POSTAGE	1	2010-12-09 16:26:00	547.32	NaN	United Kingdom
525000	538149	DOT	DOTCOM POSTAGE	1	2010-12-09 16:27:00	620.68	NaN	United Kingdom
525126	538153	DOT	DOTCOM POSTAGE	1	2010-12-09 16:31:00	822.94	NaN	United Kingdom
525147	538154	DOT	DOTCOM POSTAGE	1	2010-12-09 16:35:00	85.79	NaN	United Kingdom

3098 rows × 8 columns

```
df[(df["StockCode"].str.match("^\\d{5}$") == False) & (df["StockCode"].str.match("^\\d{5}[a-zA
```

```
array(['POST', 'D', 'DCGS0058', 'DCGS0068', 'DOT', 'M', 'DCGS0004',
       'DCGS0076', 'C2', 'BANK CHARGES', 'DCGS0003', 'TEST001',
       'gift_0001_80', 'DCGS0072', 'gift_0001_20', 'DCGS0044', 'TEST002',
       'gift_0001_10', 'gift_0001_50', 'DCGS0066N', 'gift_0001_30',
       'PADS', 'ADJUST', 'gift_0001_40', 'gift_0001_60', 'gift_0001_70',
       'gift_0001_90', 'DCGSSGIRL', 'DCGS0006', 'DCGS0016', 'DCGS0027',
       'DCGS0036', 'DCGS0039', 'DCGS0060', 'DCGS0056', 'DCGS0059', 'GIFT',
       'DCGSLBOY', 'm', 'DCGS0053', 'DCGS0062', 'DCGS0037', 'DCGSSBOY',
       'DCGSLGIRL', 'S', 'DCGS0069', 'DCGS0070', 'DCGS0075', 'B',
       'DCGS0041', 'ADJUST2', '47503J ', 'C3', 'SP1002', 'AMAZONFEE'],
      dtype=object)
```

```
df[df["StockCode"].str.contains("DOT")]
```

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
2379	489597	DOT	DOTCOM POSTAGE	1	2009-12-01 14:28:00	647.19	NaN	United Kingdom
2539	489600	DOT	DOTCOM POSTAGE	1	2009-12-01 14:43:00	55.96	NaN	United Kingdom
2551	489601	DOT	DOTCOM POSTAGE	1	2009-12-01 14:44:00	68.39	NaN	United Kingdom
2571	489602	DOT	DOTCOM POSTAGE	1	2009-12-01 14:45:00	59.35	NaN	United Kingdom
2619	489603	DOT	DOTCOM POSTAGE	1	2009-12-01 14:46:00	42.39	NaN	United Kingdom
...	...	...	...	...	...	...	...	...
524272	538071	DOT	DOTCOM POSTAGE	1	2010-12-09 14:09:00	885.94	NaN	United Kingdom
524887	538148	DOT	DOTCOM POSTAGE	1	2010-12-09 16:26:00	547.32	NaN	United Kingdom
525000	538149	DOT	DOTCOM POSTAGE	1	2010-12-09 16:27:00	620.68	NaN	United Kingdom
525126	538153	DOT	DOTCOM POSTAGE	1	2010-12-09 16:31:00	822.94	NaN	United Kingdom
525147	538154	DOT	DOTCOM POSTAGE	1	2010-12-09 16:35:00	85.79	NaN	United Kingdom

736 rows × 8 columns

```
# Simpan kode-kode aneh tadi ke dalam variabel biar gampang
kode_aneh = df[(df["StockCode"].str.match("^\\d{5}$") == False) & (df["StockCode"].str.match('

# Kita filter data cuma yang kodenya aneh, lalu kita lihat Description-nya
# .value_counts() biar sekalian tau mana yang paling sering muncul
df[df["StockCode"].isin(kode_aneh)].groupby(["StockCode", "Description"]).size().reset_index(r
```

	StockCode	Description	Jumlah Muncul
32	POST	POSTAGE	862
30	M	Manual	850
29	DOT	DOTCOM POSTAGE	735
9	C2	CARRIAGE	136
10	D	Discount	100
8	BANK CHARGES	Bank Charges	59
33	S	SAMPLES	41
2	ADJUST	Adjustment by john on 26/01/2010 16	38
16	DCGS0058	MISO PRETTY GUM	30
3	ADJUST	Adjustment by john on 26/01/2010 17	26
39	gift_0001_30	Dotcomgiftshop Gift Voucher £30.00	17
38	gift_0001_20	Dotcomgiftshop Gift Voucher £20.00	17
31	PADS	PADS TO MATCH ALL CUSHIONS	15
35	TEST001	This is a test product.	15
24	DCGS0076	SUNJAR LED NIGHT NIGHT LIGHT	13
25	DCGSSBOY	BOYS PARTY BAG	10
27	DCGSSGIRL	GIRLS PARTY BAG	10
5	AMAZONFEE	AMAZON FEE	9
11	DCGS0003	BOXED GLASS ASHTRAY	9
37	gift_0001_10	Dotcomgiftshop Gift Voucher £10.00	6
7	BANK CHARGES	Bank Charges	6
44	m	Manual	4
20	DCGS0069	OOH LA LA DOGS COLLAR	4
1	ADJUST	Adjustment by Peter on 24/05/2010 1	3
6	B	Adjust bad debt	3
4	ADJUST2	Adjustment by Peter on Jun 25 2010	3
12	DCGS0004	HAYNES CAMPER SHOULDER BAG	3
22	DCGS0072	CAT CAMOUFLAGUE COLLAR	3
19	DCGS0068	DOGS NIGHT COLLAR	2
43	gift_0001_80	Dotcomgiftshop Gift Voucher £80.00	2

	StockCode	Description	Jumlah Muncul
18	DCGS0066N	NAVY CUDDLES DOG HOODIE	2
40	gift_0001_40	Dotcomgiftshop Gift Voucher £40.00	2
34	SP1002	KID'S CHALKBOARD/EASEL	2
41	gift_0001_50	Dotcomgiftshop Gift Voucher £50.00	2
0	47503J	SET/3 FLORAL GARDEN TOOLS IN BAG	1
13	DCGS0037	KEY-RING CORKSCREW	1
17	DCGS0062	ROAD-RAGE CAR FRESHENER	1
28	DCGSSGIRL	update	1
26	DCGSSBOY	update	1
21	DCGS0070	CAMOUFLAGE DOG COLLAR	1
23	DCGS0075	CAMOUFLAGUE DOG LEAD	1
15	DCGS0044	HANDZ-OFF CAR FRESHENER	1
14	DCGS0041	HAYNES MINI-COOPER PLAYING CARDS	1
36	TEST002	This is a test product.	1
42	gift_0001_70	Dotcomgiftshop Gift Voucher £70.00	1

```python
# Kodingan ini murni dari ai, saya hanya menyuruh dia untuk membuat kodingan agar bisa memberi

# 1. Ambil lagi daftar kode aneh tadi
kode_aneh = df[(df["StockCode"].str.match("^\\d{5}$") == False) & (df["StockCode"].str.match('

# 2. Filter data cuma yang kode aneh
df_aneh = df[df["StockCode"].isin(kode_aneh)]

# 3. Kita Groupby untuk menghitung statistik Customer ID-nya
cek_customer = df_aneh.groupby("StockCode").agg(
    Total_Transaksi=('Invoice', 'count'),          # Hitung total baris
    Ada_CustomerID=('Customer ID', 'count'),       # Hitung yang ID-nya TIDAK NaN
    CustomerID_NaN=('Customer ID', lambda x: x.isna().sum()) # Hitung yang ID-nya NaN
).sort_values(by='Total_Transaksi', ascending=False)

# Tampilkan
cek_customer
```

StockCode	Total_Transaksi	Ada_CustomerID	CustomerID_NaN
POST	865	822	43
M	850	650	200
DOT	736	0	736
C2	138	125	13
D	100	97	3
ADJUST	67	61	6
BANK CHARGES	65	26	39
S	41	0	41
DCGS0058	31	0	31
gift_0001_30	21	0	21
gift_0001_20	19	0	19
TEST001	15	15	0
PADS	15	15	0
DCGS0076	13	0	13
DCGSSBOY	12	0	12
DCGSSGIRL	12	0	12
AMAZONFEE	9	0	9
DCGS0003	9	0	9
gift_0001_10	7	0	7
DCGS0004	4	0	4
DCGS0069	4	0	4
DCGS0066N	4	0	4
gift_0001_40	4	0	4
gift_0001_50	4	0	4
gift_0001_80	4	0	4
m	4	0	4
DCGS0072	3	0	3
gift_0001_70	3	0	3
B	3	0	3

StockCode	Total_Transaksi	Ada_CustomerID	CustomerID_NaN
ADJUST2	3	3	0
SP1002	3	2	1
gift_0001_60	2	0	2
gift_0001_90	2	0	2
DCGS0068	2	0	2
DCGS0062	2	0	2
DCGS0037	2	0	2
TEST002	2	1	1
DCGS0016	1	0	1
47503J	1	0	1
DCGS0059	1	0	1
DCGS0056	1	0	1
DCGS0053	1	0	1
DCGS0044	1	0	1
DCGS0039	1	0	1
DCGS0041	1	0	1
DCGS0027	1	0	1
DCGS0036	1	0	1
C3	1	0	1
DCGS0006	1	0	1
GIFT	1	0	1
DCGSLGIRL	1	0	1
DCGS0070	1	0	1
DCGS0060	1	0	1
DCGSLBOY	1	0	1
DCGS0075	1	0	1

```python
# Daftar kode yang bikin kamu penasaran
kode_penasaran = ['POST', 'M', 'DOT', 'C2', 'D', 'ADJUST', 'BANK CHARGES', 'S', 'DCGS0058']

# Loop untuk cek satu per satu
for kode in kode_penasaran:
```

```
    print(f"==================================================")
    print(f"👀 MENGINTIP ISI STOCK CODE: {kode}")
    print(f"==================================================")

    # Ambil data yang stockcode-nya sesuai
    data_intip = df[df['StockCode'] == kode]

    # Tampilkan 10 baris pertama saja biar gak penuh layarnya
    display(data_intip.head(10))

    # Kasih jarak enter biar enak bacanya
    print("\n")
```

```
==================================================
👀 MENGINTIP ISI STOCK CODE: POST
==================================================
```

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
89	489439	POST	POSTAGE	3	2009-12-01 09:28:00	18.00	12682.00	France
126	489444	POST	POSTAGE	1	2009-12-01 09:55:00	141.00	12636.00	USA
173	489447	POST	POSTAGE	1	2009-12-01 10:10:00	130.00	12362.00	Belgium
625	489526	POST	POSTAGE	6	2009-12-01 11:50:00	18.00	12533.00	Germany
927	C489538	POST	POSTAGE	-1	2009-12-01 12:18:00	9.58	15796.00	United Kingdom
1244	489557	POST	POSTAGE	4	2009-12-01 12:52:00	18.00	12490.00	France
3451	C489685	POST	POSTAGE	-1	2009-12-02 10:28:00	18.00	12523.00	France
6406	489883	POST	POSTAGE	3	2009-12-02 16:24:00	18.00	12437.00	France
9103	C490117	POST	POSTAGE	-1	2009-12-03 17:38:00	2.99	16570.00	United Kingdom
9153	C490120	POST	POSTAGE	-2	2009-12-03 17:52:00	18.00	14277.00	France

```
==================================================
👀 MENGINTIP ISI STOCK CODE: M
==================================================
```

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
2697	489609	M	Manual	1	2009-12-01 14:50:00	4.00	NaN	United Kingdom
3053	C489651	M	Manual	-1	2009-12-01 16:48:00	5.10	17804.00	United Kingdom
5897	C489859	M	Manual	-1	2009-12-02 14:45:00	69.57	NaN	United Kingdom
9259	C490126	M	Manual	-1	2009-12-03 18:12:00	5.95	15884.00	United Kingdom
9307	C490129	M	Manual	-1	2009-12-03 18:26:00	1998.49	15482.00	United Kingdom
11310	490300	M	Manual	1	2009-12-04 14:19:00	0.85	12970.00	United Kingdom
11311	490300	M	Manual	1	2009-12-04 14:19:00	0.21	12970.00	United Kingdom
16107	490727	M	Manual	1	2009-12-07 16:38:00	0.00	17231.00	United Kingdom
17273	C490748	M	Manual	-1	2009-12-07 18:14:00	309.73	12748.00	United Kingdom
17386	490760	M	Manual	1	2009-12-08 09:49:00	10.00	14295.00	United Kingdom

```
==================================================
👀 MENGINTIP ISI STOCK CODE: DOT
==================================================
```

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
2379	489597	DOT	DOTCOM POSTAGE	1	2009-12-01 14:28:00	647.19	NaN	United Kingdom
2539	489600	DOT	DOTCOM POSTAGE	1	2009-12-01 14:43:00	55.96	NaN	United Kingdom
2551	489601	DOT	DOTCOM POSTAGE	1	2009-12-01 14:44:00	68.39	NaN	United Kingdom
2571	489602	DOT	DOTCOM POSTAGE	1	2009-12-01 14:45:00	59.35	NaN	United Kingdom
2619	489603	DOT	DOTCOM POSTAGE	1	2009-12-01 14:46:00	42.39	NaN	United Kingdom
2644	489604	DOT	DOTCOM POSTAGE	1	2009-12-01 14:47:00	50.87	NaN	United Kingdom
2686	489607	DOT	DOTCOM POSTAGE	1	2009-12-01 14:49:00	76.30	NaN	United Kingdom
2698	489609	DOT	DOTCOM POSTAGE	1	2009-12-01 14:50:00	74.61	NaN	United Kingdom
2771	489612	DOT	DOTCOM POSTAGE	1	2009-12-01 14:55:00	87.39	NaN	United Kingdom
2806	489614	DOT	DOTCOM POSTAGE	1	2009-12-01 14:56:00	80.30	NaN	United Kingdom

```
==================================================
👀 MENGINTIP ISI STOCK CODE: C2
==================================================
```

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
9292	490127	C2	CARRIAGE	1	2009-12-03 18:13:00	50.00	14156.00	EIRE
14481	490541	C2	CARRIAGE	1	2009-12-07 09:25:00	50.00	NaN	EIRE
14502	490542	C2	CARRIAGE	1	2009-12-07 09:42:00	50.00	14911.00	EIRE
19541	490998	C2	CARRIAGE	1	2009-12-08 17:24:00	50.00	16253.00	United Kingdom
22803	491160	C2	CARRIAGE	1	2009-12-10 10:29:00	50.00	14911.00	EIRE
27494	491702	C2	CARRIAGE	1	2009-12-13 13:53:00	50.00	NaN	EIRE
32207	491990	C2	NaN	100	2009-12-15 10:06:00	0.00	NaN	United Kingdom
32964	492092	C2	CARRIAGE	1	2009-12-15 14:03:00	50.00	14156.00	EIRE
34330	492250	C2	CARRIAGE	1	2009-12-16 10:45:00	50.00	18286.00	United Kingdom
39877	492746	C2	CARRIAGE	1	2009-12-18 13:01:00	50.00	NaN	EIRE

```
==================================================
👀 MENGINTIP ISI STOCK CODE: D
==================================================
```

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
735	C489535	D	Discount	-1	2009-12-01 12:11:00	9.00	15299.00	United Kingdom
736	C489535	D	Discount	-1	2009-12-01 12:11:00	19.00	15299.00	United Kingdom
24675	C491428	D	Discount	-1	2009-12-10 20:23:00	9.10	15494.00	United Kingdom
29414	C491845	D	Discount	-1	2009-12-14 14:54:00	1.59	NaN	United Kingdom
29958	C491962	D	Discount	-1	2009-12-14 16:38:00	0.59	13206.00	United Kingdom
39127	C492693	D	Discount	-1	2009-12-17 18:47:00	6.85	13408.00	United Kingdom
44782	C493373	D	Discount	-1	2009-12-23 11:22:00	64.37	15044.00	United Kingdom
62231	C494909	D	Discount	-30	2010-01-19 16:45:00	0.40	12931.00	United Kingdom
62232	C494909	D	Discount	-30	2010-01-19 16:45:00	0.13	12931.00	United Kingdom
62962	C494984	D	Discount	-1	2010-01-20 11:12:00	70.00	17949.00	United Kingdom

```
==================================================
👀 MENGINTIP ISI STOCK CODE: ADJUST
==================================================
```

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
70975	495732	ADJUST	Adjustment by john on 26/01/2010 16	1	2010-01-26 16:20:00	96.46	NaN	EIRE
70976	495733	ADJUST	Adjustment by john on 26/01/2010 16	1	2010-01-26 16:21:00	68.34	14911.00	EIRE
70977	495735	ADJUST	Adjustment by john on 26/01/2010 16	1	2010-01-26 16:22:00	201.56	12745.00	EIRE
70978	495734	ADJUST	Adjustment by john on 26/01/2010 16	1	2010-01-26 16:22:00	205.82	14911.00	EIRE
70979	C495737	ADJUST	Adjustment by john on 26/01/2010 16	-1	2010-01-26 16:23:00	10.50	16154.00	United Kingdom
70980	495736	ADJUST	Adjustment by john on 26/01/2010 16	1	2010-01-26 16:23:00	21.00	12606.00	Spain
70981	C495740	ADJUST	Adjustment by john on 26/01/2010 16	-1	2010-01-26 16:24:00	14.00	13054.00	United Kingdom
70982	C495738	ADJUST	Adjustment by john on 26/01/2010 16	-1	2010-01-26 16:24:00	26.25	12454.00	Spain
70983	C495739	ADJUST	Adjustment by john on 26/01/2010 16	-1	2010-01-26 16:24:00	10.50	15383.00	United Kingdom
70984	C495744	ADJUST	Adjustment by john on 26/01/2010 16	-1	2010-01-26 16:25:00	91.89	12706.00	Finland

```
==================================================
👀 MENGINTIP ISI STOCK CODE: BANK CHARGES
==================================================
```

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
18410	C490943	BANK CHARGES	Bank Charges	-1	2009-12-08 14:08:00	15.00	16703.00	United Kingdom
18466	490948	BANK CHARGES	Bank Charges	1	2009-12-08 14:29:00	15.00	16805.00	United Kingdom
33435	C492206	BANK CHARGES	Bank Charges	-1	2009-12-15 16:32:00	848.43	NaN	United Kingdom
55948	C494438	BANK CHARGES	Bank Charges	-1	2010-01-14 12:15:00	767.99	NaN	United Kingdom
94431	498269	BANK CHARGES	Bank Charges	1	2010-02-17 15:03:00	15.00	16928.00	United Kingdom
104220	C499374	BANK CHARGES	Bank Charges	-1	2010-02-26 11:55:00	467.54	NaN	United Kingdom
114180	C500319	BANK CHARGES	Bank Charges	-11	2010-03-07 12:02:00	0.96	NaN	United Kingdom
115208	C500352	BANK CHARGES	Bank Charges	-1	2010-03-07 15:08:00	11.29	NaN	United Kingdom
118558	C500708	BANK CHARGES	Bank Charges	-1	2010-03-09 13:56:00	372.30	NaN	United Kingdom
136411	C502459	BANK CHARGES	Bank Charges	-1	2010-03-24 14:25:00	39.24	NaN	United Kingdom

```
==================================================
👀 MENGINTIP ISI STOCK CODE: S
==================================================
```

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
114061	C500305	S	SAMPLES	-1	2010-03-07 10:59:00	73.80	NaN	United Kingdom
114083	C500309	S	SAMPLES	-1	2010-03-07 11:09:00	32.03	NaN	United Kingdom
133558	C502083	S	SAMPLES	-1	2010-03-22 15:50:00	170.37	NaN	United Kingdom
133582	C502088	S	SAMPLES	-1	2010-03-22 16:03:00	259.59	NaN	United Kingdom
136253	C502438	S	SAMPLES	-1	2010-03-24 13:11:00	605.18	NaN	United Kingdom
136259	C502442	S	SAMPLES	-1	2010-03-24 13:20:00	94.19	NaN	United Kingdom
181508	506601	S	SAMPLES	1	2010-04-30 14:49:00	73.80	NaN	United Kingdom
181509	C506602	S	SAMPLES	-1	2010-04-30 14:56:00	3.84	NaN	United Kingdom
181510	C506602	S	SAMPLES	-1	2010-04-30 14:56:00	3.55	NaN	United Kingdom
181511	C506602	S	SAMPLES	-1	2010-04-30 14:56:00	77.00	NaN	United Kingdom

```
==================================================
👀 MENGINTIP ISI STOCK CODE: DCGS0058
==================================================
```

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
2377	489597	DCGS0058	MISO PRETTY GUM	1	2009-12-01 14:28:00	0.83	NaN	United Kingdom
8372	490074	DCGS0058	MISO PRETTY GUM	1	2009-12-03 14:39:00	0.83	NaN	United Kingdom
17264	490745	DCGS0058	MISO PRETTY GUM	1	2009-12-07 18:02:00	0.83	NaN	United Kingdom
30671	491969	DCGS0058	MISO PRETTY GUM	1	2009-12-14 17:57:00	0.83	NaN	United Kingdom
31652	491970	DCGS0058	MISO PRETTY GUM	1	2009-12-14 18:03:00	0.83	NaN	United Kingdom
32045	491971	DCGS0058	MISO PRETTY GUM	2	2009-12-14 18:37:00	0.83	NaN	United Kingdom
34668	492303	DCGS0058	MISO PRETTY GUM	1	2009-12-16 11:57:00	0.83	NaN	United Kingdom
37222	492425	DCGS0058	MISO PRETTY GUM	1	2009-12-16 17:58:00	0.83	NaN	United Kingdom
40878	492782	DCGS0058	MISO PRETTY GUM	1	2009-12-18 17:06:00	0.83	NaN	United Kingdom
41260	492783	DCGS0058	MISO PRETTY GUM	2	2009-12-18 17:15:00	0.83	NaN	United Kingdom

```
cleaned_df = df.copy()
```

```
cleaned_df["Invoice"] = cleaned_df["Invoice"].astype("str")

mask = (
    cleaned_df["Invoice"].str.match("^\\d{6}$") == True
)

cleaned_df= cleaned_df[mask]
cleaned_df
```

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
0	489434	85048	15CM CHRISTMAS GLASS BALL 20 LIGHTS	12	2009-12-01 07:45:00	6.95	13085.00	United Kingdom
1	489434	79323P	PINK CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13085.00	United Kingdom
2	489434	79323W	WHITE CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13085.00	United Kingdom
3	489434	22041	RECORD FRAME 7" SINGLE SIZE	48	2009-12-01 07:45:00	2.10	13085.00	United Kingdom
4	489434	21232	STRAWBERRY CERAMIC TRINKET BOX	24	2009-12-01 07:45:00	1.25	13085.00	United Kingdom
...	...	...	...	...	...	...	...	...
525456	538171	22271	FELTCRAFT DOLL ROSIE	2	2010-12-09 20:01:00	2.95	17530.00	United Kingdom
525457	538171	22750	FELTCRAFT PRINCESS LOLA DOLL	1	2010-12-09 20:01:00	3.75	17530.00	United Kingdom
525458	538171	22751	FELTCRAFT PRINCESS OLIVIA DOLL	1	2010-12-09 20:01:00	3.75	17530.00	United Kingdom
525459	538171	20970	PINK FLORAL FELTCRAFT SHOULDER BAG	2	2010-12-09 20:01:00	3.75	17530.00	United Kingdom
525460	538171	21931	JUMBO STORAGE BAG SUKI	2	2010-12-09 20:01:00	1.95	17530.00	United Kingdom

515252 rows × 8 columns

```
cleaned_df["StockCode"] = cleaned_df["StockCode"].astype("str")

mask = (
    (cleaned_df["StockCode"].str.match("^\\d{5}") == True)
    | (cleaned_df["StockCode"].str.match("^\\d{5}[a-zA-Z]+$") == True)
    | (cleaned_df["StockCode"].str.match("PADS") == True)
)

cleaned_df= cleaned_df[mask]
cleaned_df
```

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
0	489434	85048	15CM CHRISTMAS GLASS BALL 20 LIGHTS	12	2009-12-01 07:45:00	6.95	13085.00	United Kingdom
1	489434	79323P	PINK CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13085.00	United Kingdom
2	489434	79323W	WHITE CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13085.00	United Kingdom
3	489434	22041	RECORD FRAME 7" SINGLE SIZE	48	2009-12-01 07:45:00	2.10	13085.00	United Kingdom
4	489434	21232	STRAWBERRY CERAMIC TRINKET BOX	24	2009-12-01 07:45:00	1.25	13085.00	United Kingdom
...	...	...	...	...	...	...	...	...
525456	538171	22271	FELTCRAFT DOLL ROSIE	2	2010-12-09 20:01:00	2.95	17530.00	United Kingdom
525457	538171	22750	FELTCRAFT PRINCESS LOLA DOLL	1	2010-12-09 20:01:00	3.75	17530.00	United Kingdom
525458	538171	22751	FELTCRAFT PRINCESS OLIVIA DOLL	1	2010-12-09 20:01:00	3.75	17530.00	United Kingdom
525459	538171	20970	PINK FLORAL FELTCRAFT SHOULDER BAG	2	2010-12-09 20:01:00	3.75	17530.00	United Kingdom
525460	538171	21931	JUMBO STORAGE BAG SUKI	2	2010-12-09 20:01:00	1.95	17530.00	United Kingdom

512797 rows × 8 columns

```
cleaned_df.describe()
```

	Quantity	InvoiceDate	Price	Customer ID
count	512797.00	512797	512797.00	406337.00
mean	11.00	2010-06-28 18:26:53.830657792	3.39	15373.63
min	-9600.00	2009-12-01 07:45:00	0.00	12346.00
25%	1.00	2010-03-21 13:27:00	1.25	14004.00
50%	3.00	2010-07-06 14:25:00	2.10	15326.00
75%	10.00	2010-10-15 14:50:00	4.21	16814.00
max	19152.00	2010-12-09 20:01:00	1157.15	18287.00
std	104.35	NaN	5.07	1677.37

```
cleaned_df.dropna(subset=["Customer ID"], inplace=True)
```

```
C:\Users\MSI KATANA 15\AppData\Local\Temp\ipykernel_18036\1633333693.py:1: SettingWithCopyWar
ning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guid
e/indexing.html#returning-a-view-versus-a-copy
  cleaned_df.dropna(subset=["Customer ID"], inplace=True)
```

```
print("--- BEFORE ---")
df.describe()
```

```
--- BEFORE ---
```

	Quantity	InvoiceDate	Price	Customer ID
count	525461.00	525461	525461.00	417534.00
mean	10.34	2010-06-28 11:37:36.845017856	4.69	15360.65
min	-9600.00	2009-12-01 07:45:00	-53594.36	12346.00
25%	1.00	2010-03-21 12:20:00	1.25	13983.00
50%	3.00	2010-07-06 09:51:00	2.10	15311.00
75%	10.00	2010-10-15 12:45:00	4.21	16799.00
max	19152.00	2010-12-09 20:01:00	25111.09	18287.00
std	107.42	NaN	146.13	1680.81

```
print("--- AFTER ---")
cleaned_df.describe()
```

```
--- AFTER ---
```

	Quantity	InvoiceDate	Price	Customer ID
count	406337.00	406337	406337.00	406337.00
mean	13.62	2010-07-01 10:11:06.543288320	2.99	15373.63
min	1.00	2009-12-01 07:45:00	0.00	12346.00
25%	2.00	2010-03-26 14:01:00	1.25	14004.00
50%	5.00	2010-07-09 15:48:00	1.95	15326.00
75%	12.00	2010-10-14 17:09:00	3.75	16814.00
max	19152.00	2010-12-09 20:01:00	295.00	18287.00
std	97.00	NaN	4.29	1677.37

```
cleaned_df[cleaned_df["Price"]== 0].head(5)
```

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
4674	489825	22076	6 RIBBONS EMPIRE	12	2009-12-02 13:34:00	0.00	16126.00	United Kingdom
6781	489998	48185	DOOR MAT FAIRY CAKE	2	2009-12-03 11:19:00	0.00	15658.00	United Kingdom
18738	490961	22065	CHRISTMAS PUDDING TRINKET POT	1	2009-12-08 15:25:00	0.00	14108.00	United Kingdom
18739	490961	22142	CHRISTMAS CRAFT WHITE FAIRY	12	2009-12-08 15:25:00	0.00	14108.00	United Kingdom
32916	492079	85042	ANTIQUE LILY FAIRY LIGHTS	8	2009-12-15 13:49:00	0.00	15070.00	United Kingdom

```
len(cleaned_df[cleaned_df["Price"]== 0])
```

```
28
```

```
cleaned_df = cleaned_df[cleaned_df["Price"]> 0 ]
```

```
cleaned_df.describe()
```

	Quantity	InvoiceDate	Price	Customer ID
count	406309.00	406309	406309.00	406309.00
mean	13.62	2010-07-01 10:14:25.869572352	2.99	15373.72
min	1.00	2009-12-01 07:45:00	0.00	12346.00
25%	2.00	2010-03-26 14:01:00	1.25	14006.00
50%	5.00	2010-07-09 15:48:00	1.95	15326.00
75%	12.00	2010-10-14 17:09:00	3.75	16814.00
max	19152.00	2010-12-09 20:01:00	295.00	18287.00
std	97.00	NaN	4.29	1677.33

```
cleaned_df["Price"].min()
```

```
np.float64(0.001)
```

```
len(cleaned_df)/len(df) # Good Practise untuk mengetahu berapa banyak data yang hilang / kamu
```

```
0.7732429238325965
```

```
# Simpan data yang sudah bersih ke file CSV baru
# index=False biar nomor baris (0, 1, 2...) gak ikut kesimpen jadi kolom baru
cleaned_df.to_csv("../data/online_retail_clean.csv", index=False)
```

```
cleaned_df["SalesLineTotal"] = cleaned_df["Quantity"] * cleaned_df["Price"]

cleaned_df
```

```
C:\Users\MSI KATANA 15\AppData\Local\Temp\ipykernel_18036\2846558921.py:1: SettingWithCopyWar
ning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guid
e/indexing.html#returning-a-view-versus-a-copy
  cleaned_df["SalesLineTotal"] = cleaned_df["Quantity"] * cleaned_df["Price"]
```

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country	SalesLine
0	489434	85048	15CM CHRISTMAS GLASS BALL 20 LIGHTS	12	2009-12-01 07:45:00	6.95	13085.00	United Kingdom	
1	489434	79323P	PINK CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13085.00	United Kingdom	
2	489434	79323W	WHITE CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13085.00	United Kingdom	
3	489434	22041	RECORD FRAME 7" SINGLE SIZE	48	2009-12-01 07:45:00	2.10	13085.00	United Kingdom	1
4	489434	21232	STRAWBERRY CERAMIC TRINKET BOX	24	2009-12-01 07:45:00	1.25	13085.00	United Kingdom	
...	...	...	...	...	...	...	...	...	
525456	538171	22271	FELTCRAFT DOLL ROSIE	2	2010-12-09 20:01:00	2.95	17530.00	United Kingdom	
525457	538171	22750	FELTCRAFT PRINCESS LOLA DOLL	1	2010-12-09 20:01:00	3.75	17530.00	United Kingdom	
525458	538171	22751	FELTCRAFT PRINCESS OLIVIA DOLL	1	2010-12-09 20:01:00	3.75	17530.00	United Kingdom	
525459	538171	20970	PINK FLORAL FELTCRAFT SHOULDER BAG	2	2010-12-09 20:01:00	3.75	17530.00	United Kingdom	
525460	538171	21931	JUMBO STORAGE BAG SUKI	2	2010-12-09 20:01:00	1.95	17530.00	United Kingdom	

406309 rows × 9 columns

```python
aggregated_df = cleaned_df.groupby(by="Customer ID", as_index=False) \
    .agg(
        MonetaryValue = ("SalesLineTotal", "sum"),
        Frequency = ("Invoice", "nunique"),
        LastInvoiceDate = ("InvoiceDate", "max")
    )
```

```
aggregated_df.head(10)
```

	Customer ID	MonetaryValue	Frequency	LastInvoiceDate
0	12346.00	169.36	2	2010-06-28 13:53:00
1	12347.00	1323.32	2	2010-12-07 14:57:00
2	12348.00	221.16	1	2010-09-27 14:59:00
3	12349.00	2221.14	2	2010-10-28 08:23:00
4	12351.00	300.93	1	2010-11-29 15:23:00
5	12352.00	343.80	2	2010-11-29 10:07:00
6	12353.00	317.76	1	2010-10-27 12:44:00
7	12355.00	488.21	1	2010-05-21 11:59:00
8	12356.00	3126.25	3	2010-11-24 12:24:00
9	12357.00	11229.99	1	2010-11-16 10:05:00

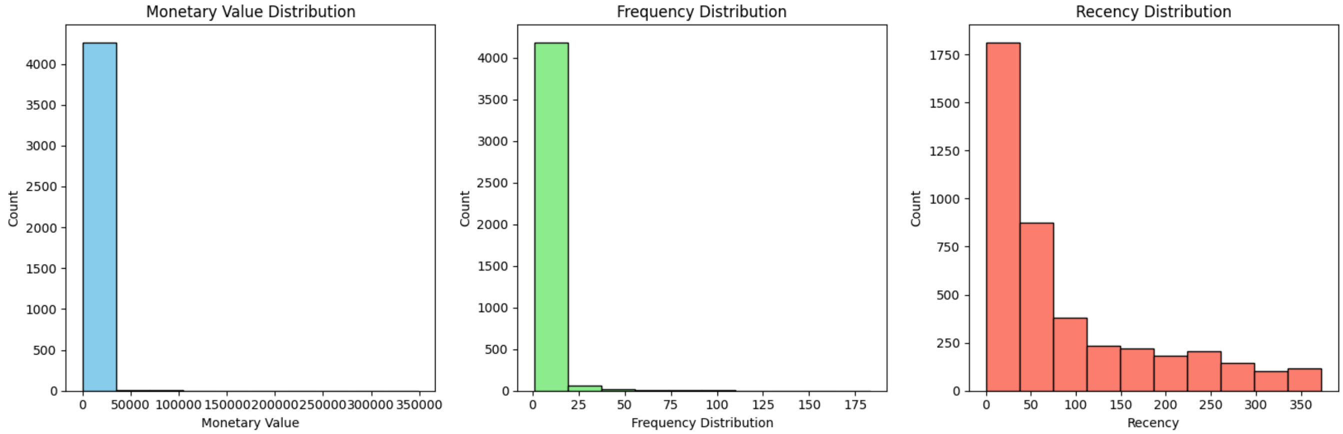
```
aggregated_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4285 entries, 0 to 4284
Data columns (total 4 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   Customer ID      4285 non-null   float64
 1   MonetaryValue    4285 non-null   float64
 2   Frequency        4285 non-null   int64
 3   LastInvoiceDate  4285 non-null   datetime64[ns]
dtypes: datetime64[ns](1), float64(2), int64(1)
memory usage: 134.0 KB
```

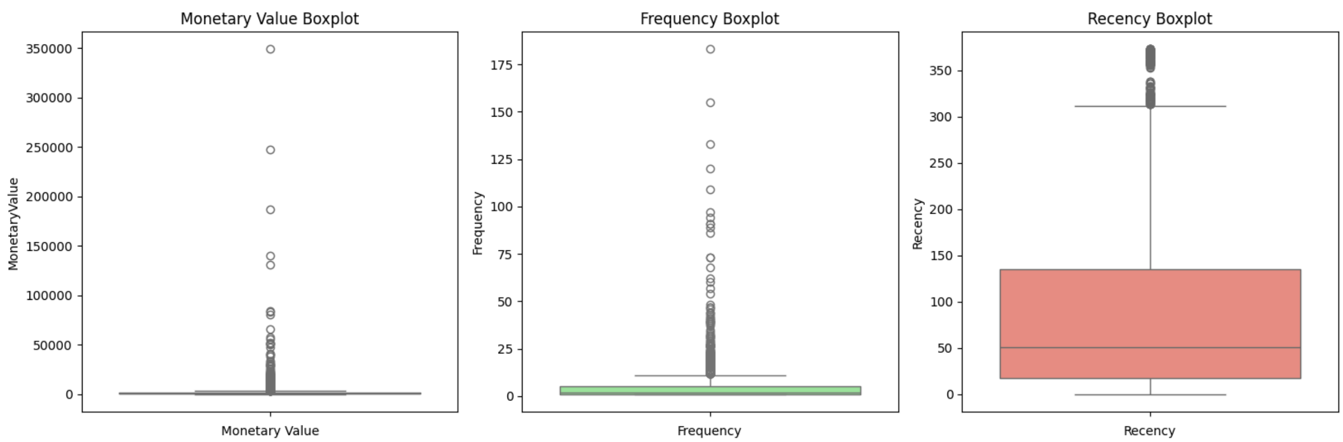
```
max_invoice_date = aggregated_df["LastInvoiceDate"].max()

aggregated_df["Recency"] = (max_invoice_date - aggregated_df["LastInvoiceDate"]).dt.days

aggregated_df.head(10)
```

	Customer ID	MonetaryValue	Frequency	LastInvoiceDate	Recency
0	12346.00	169.36	2	2010-06-28 13:53:00	164
1	12347.00	1323.32	2	2010-12-07 14:57:00	2
2	12348.00	221.16	1	2010-09-27 14:59:00	73
3	12349.00	2221.14	2	2010-10-28 08:23:00	42
4	12351.00	300.93	1	2010-11-29 15:23:00	10
5	12352.00	343.80	2	2010-11-29 10:07:00	10
6	12353.00	317.76	1	2010-10-27 12:44:00	43
7	12355.00	488.21	1	2010-05-21 11:59:00	202
8	12356.00	3126.25	3	2010-11-24 12:24:00	15
9	12357.00	11229.99	1	2010-11-16 10:05:00	23

```python
plt.figure(figsize=(15,5))

plt.subplot(1,3,1)
plt.hist(aggregated_df['MonetaryValue'], bins=10, color='skyblue', edgecolor='black')
plt.title('Monetary Value Distribution')
plt.xlabel('Monetary Value')
plt.ylabel('Count')

plt.subplot(1,3,2)
plt.hist(aggregated_df['Frequency'], bins=10, color='lightgreen', edgecolor='black')
plt.title('Frequency Distribution')
plt.xlabel('Frequency Distribution')
plt.ylabel('Count')

plt.subplot(1,3,3)
plt.hist(aggregated_df['Recency'], bins=10, color='salmon', edgecolor='black')
plt.title('Recency Distribution')
plt.xlabel('Recency')
plt.ylabel('Count')

plt.tight_layout()
plt.show()
```




```python
# Metode paling cocok buat ngecek Outliers
plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
sns.boxplot(data=aggregated_df['MonetaryValue'], color='skyblue')
plt.title('Monetary Value Boxplot')
plt.xlabel('Monetary Value')

plt.subplot(1, 3, 2)
sns.boxplot(data=aggregated_df['Frequency'], color='lightgreen')
plt.title('Frequency Boxplot')
plt.xlabel('Frequency')

plt.subplot(1, 3, 3)
sns.boxplot(data=aggregated_df['Recency'], color='salmon')
plt.title('Recency Boxplot')
plt.xlabel('Recency')

plt.tight_layout()
plt.show()
```




```python
# Disini saya meminta tolong kepada AI untuk "ah i see, terus ini bisa gak yaa gambar itu dita

# Fungsi untuk bikin Tabel Distribusi Frekuensi
def buat_tabel_distribusi(kolom, nama_kolom, n_bins=10):
    # 1. Bikin rentang (binning)
    # pd.cut memotong data jadi 10 bagian
    counts = pd.cut(aggregated_df[kolom], bins=n_bins).value_counts().sort_index()
```

```
    # 2. Bikin DataFrame rapi
    tabel = pd.DataFrame({
        'Rentang Nilai': counts.index,
        'Jumlah Customer': counts.values
    })

    # 3. Hitung Persentase biar lebih kebayang
    tabel['Persentase (%)'] = (tabel['Jumlah Customer'] / tabel['Jumlah Customer'].sum()) * 10

    print(f"--- TABEL DISTRIBUSI: {nama_kolom} ---")
    display(tabel)
    print("\n")

# Panggil fungsinya buat 3 kolom RFM
buat_tabel_distribusi('MonetaryValue', 'MONETARY (Total Belanja)')
buat_tabel_distribusi('Frequency', 'FREQUENCY (Seringnya Belanja)')
buat_tabel_distribusi('Recency', 'RECENCY (Hari Terakhir Belanja)')
```

```
--- TABEL DISTRIBUSI: MONETARY (Total Belanja) ---
```

	Rentang Nilai	Jumlah Customer	Persentase (%)
0	(-347.613, 34917.83]	4265	99.53
1	(34917.83, 69834.11]	12	0.28
2	(69834.11, 104750.39]	3	0.07
3	(104750.39, 139666.67]	1	0.02
4	(139666.67, 174582.95]	1	0.02
5	(174582.95, 209499.23]	1	0.02
6	(209499.23, 244415.51]	0	0.00
7	(244415.51, 279331.79]	1	0.02
8	(279331.79, 314248.07]	0	0.00
9	(314248.07, 349164.35]	1	0.02

```
--- TABEL DISTRIBUSI: FREQUENCY (Seringnya Belanja) ---
```

	Rentang Nilai	Jumlah Customer	Persentase (%)
0	(0.818, 19.2]	4187	97.71
1	(19.2, 37.4]	62	1.45
2	(37.4, 55.6]	19	0.44
3	(55.6, 73.8]	6	0.14
4	(73.8, 92.0]	4	0.09
5	(92.0, 110.2]	3	0.07
6	(110.2, 128.4]	1	0.02
7	(128.4, 146.6]	1	0.02
8	(146.6, 164.8]	1	0.02
9	(164.8, 183.0]	1	0.02

```
--- TABEL DISTRIBUSI: RECENCY (Hari Terakhir Belanja) ---
```

	Rentang Nilai	Jumlah Customer	Persentase (%)
0	(-0.373, 37.3]	1814	42.33
1	(37.3, 74.6]	876	20.44
2	(74.6, 111.9]	382	8.91
3	(111.9, 149.2]	234	5.46
4	(149.2, 186.5]	221	5.16
5	(186.5, 223.8]	184	4.29
6	(223.8, 261.1]	208	4.85
7	(261.1, 298.4]	145	3.38
8	(298.4, 335.7]	104	2.43
9	(335.7, 373.0]	117	2.73

```
M_Q1 = aggregated_df["MonetaryValue"].quantile(0.25)
M_Q3 = aggregated_df["MonetaryValue"].quantile(0.75)
M_IQR = M_Q3 - M_Q1

monetary_outliers_df = aggregated_df[(aggregated_df["MonetaryValue"]> (M_Q3 + 1.5 * M_IQR)) |
monetary_outliers_df.describe()
```

	Customer ID	MonetaryValue	Frequency	LastInvoiceDate	Recency
count	423.00	423.00	423.00	423	423.00
mean	15103.04	12188.10	17.17	2010-11-09 12:26:02.978723328	30.04
min	12357.00	3802.04	1.00	2009-12-10 18:03:00	0.00
25%	13622.00	4605.94	8.00	2010-11-08 13:17:30	3.00
50%	14961.00	6191.32	12.00	2010-11-26 12:19:00	13.00
75%	16692.00	10273.24	18.00	2010-12-06 10:34:30	31.00
max	18260.00	349164.35	183.00	2010-12-09 19:32:00	364.00
std	1728.66	25830.85	19.73	NaN	51.54

```
F_Q1 = aggregated_df["Frequency"].quantile(0.25)
F_Q3 = aggregated_df["Frequency"].quantile(0.75)
F_IQR = F_Q3 - F_Q1

frequency_outliers_df = aggregated_df[(aggregated_df["Frequency"]> (F_Q3 + 1.5 * F_IQR)) | (ag
frequency_outliers_df.describe()
```

	Customer ID	MonetaryValue	Frequency	LastInvoiceDate	Recency
count	279.00	279.00	279.00	279	279.00
mean	15352.66	14409.71	23.81	2010-11-23 11:06:20.645161216	16.09
min	12437.00	1094.39	12.00	2010-05-12 16:51:00	0.00
25%	13800.00	4331.56	13.00	2010-11-20 13:14:30	2.00
50%	15465.00	6615.77	17.00	2010-12-02 10:46:00	7.00
75%	16828.50	11692.41	23.00	2010-12-07 11:08:30	19.00
max	18260.00	349164.35	183.00	2010-12-09 19:32:00	211.00
std	1748.43	31381.74	21.93	NaN	26.59

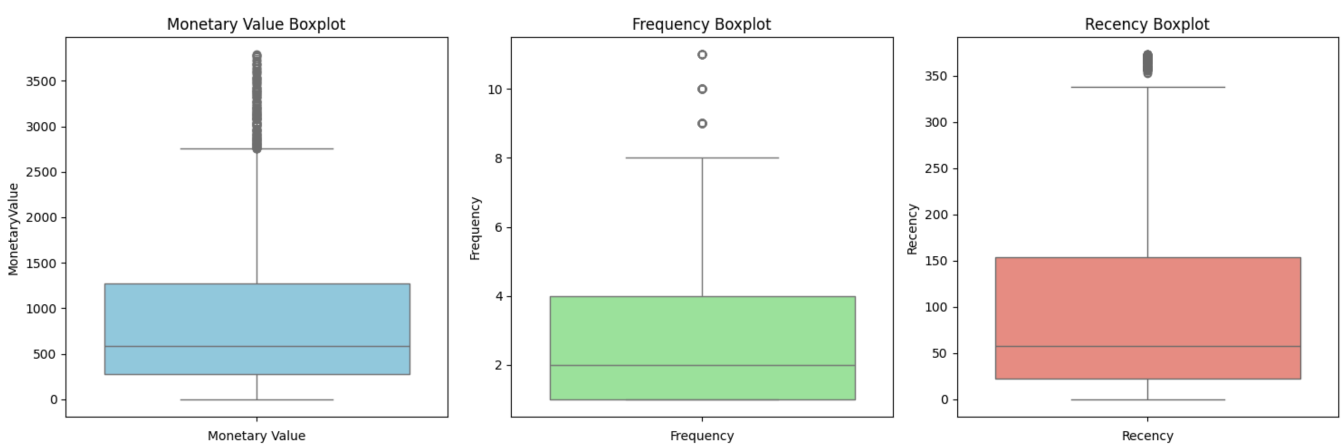
```
# Mencari Customer ID yang ada di KEDUA tabel outlier
overlap_customers = set(monetary_outliers_df['Customer ID']).intersection(set(frequency_outlie

print(f"Jumlah Sultan (Monetary Outlier): {len(monetary_outliers_df)}")
print(f"Jumlah Si Rajin (Frequency Outlier): {len(frequency_outliers_df)}")
print(f"Jumlah 'Sultan Rajin' (Overlap): {len(overlap_customers)}")
```

```
Jumlah Sultan (Monetary Outlier): 423
Jumlah Si Rajin (Frequency Outlier): 279
Jumlah 'Sultan Rajin' (Overlap): 226
```

```
non_outliers_df = aggregated_df[(~aggregated_df.index.isin(monetary_outliers_df.index)) & (~ag

non_outliers_df.describe()
```

	Customer ID	MonetaryValue	Frequency	LastInvoiceDate	Recency
count	3809.00	3809.00	3809.00	3809	3809.00
mean	15376.48	885.50	2.86	2010-09-03 11:16:46.516146176	97.08
min	12346.00	1.55	1.00	2009-12-01 10:49:00	0.00
25%	13912.00	279.91	1.00	2010-07-08 14:48:00	22.00
50%	15389.00	588.05	2.00	2010-10-12 16:25:00	58.00
75%	16854.00	1269.05	4.00	2010-11-17 13:14:00	154.00
max	18287.00	3788.21	11.00	2010-12-09 20:01:00	373.00
std	1693.20	817.67	2.24	NaN	98.11

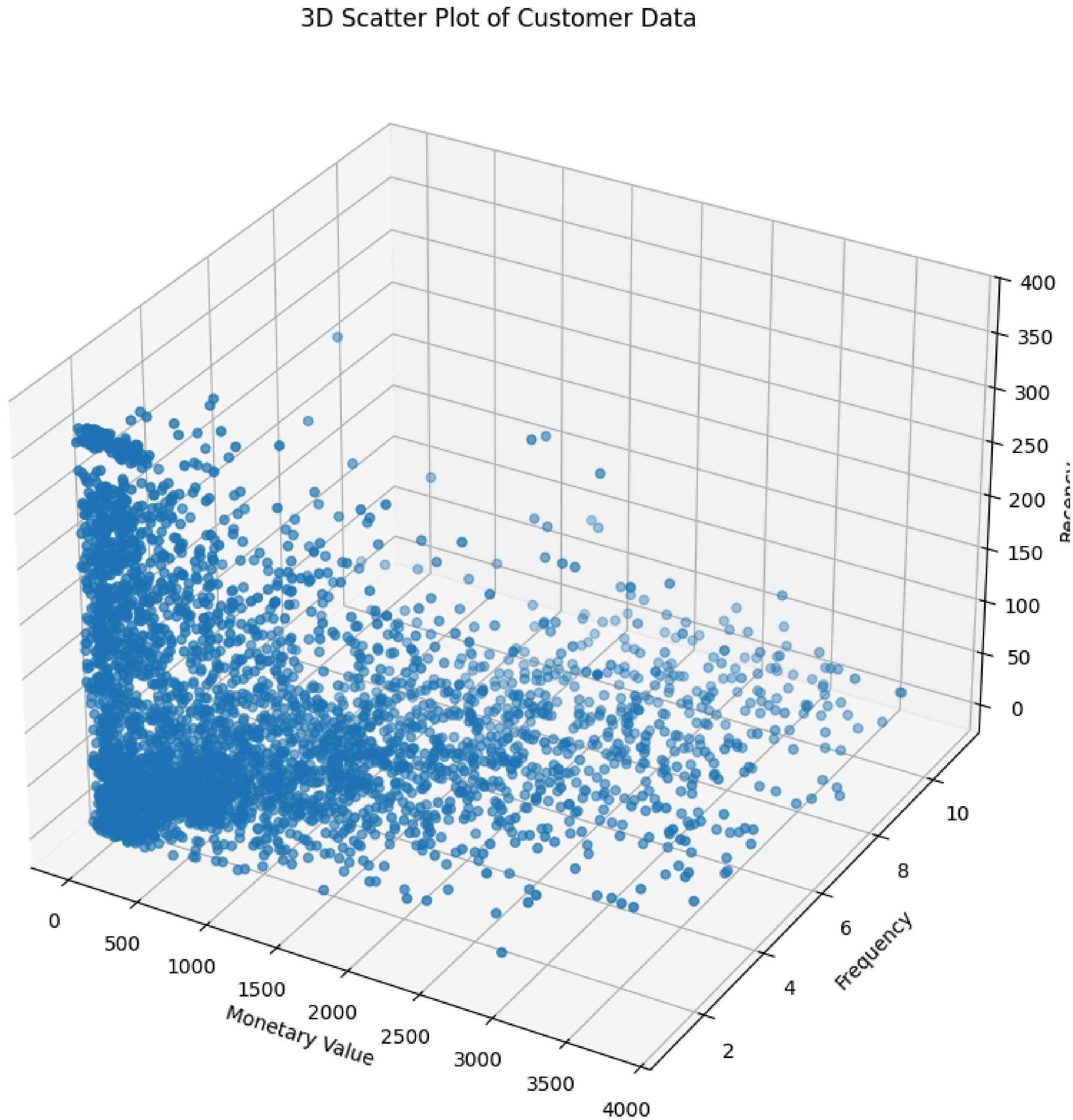
```
# Metode paling cocok buat ngecek Outliers
plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
sns.boxplot(data=non_outliers_df['MonetaryValue'], color='skyblue')
plt.title('Monetary Value Boxplot')
plt.xlabel('Monetary Value')

plt.subplot(1, 3, 2)
sns.boxplot(data=non_outliers_df['Frequency'], color='lightgreen')
plt.title('Frequency Boxplot')
plt.xlabel('Frequency')

plt.subplot(1, 3, 3)
sns.boxplot(data=non_outliers_df['Recency'], color='salmon')
plt.title('Recency Boxplot')
plt.xlabel('Recency')

plt.tight_layout()
plt.show()
```




```
non_outliers_df.head(10)
```

	Customer ID	MonetaryValue	Frequency	LastInvoiceDate	Recency
0	12346.00	169.36	2	2010-06-28 13:53:00	164
1	12347.00	1323.32	2	2010-12-07 14:57:00	2
2	12348.00	221.16	1	2010-09-27 14:59:00	73
3	12349.00	2221.14	2	2010-10-28 08:23:00	42
4	12351.00	300.93	1	2010-11-29 15:23:00	10
5	12352.00	343.80	2	2010-11-29 10:07:00	10
6	12353.00	317.76	1	2010-10-27 12:44:00	43
7	12355.00	488.21	1	2010-05-21 11:59:00	202
8	12356.00	3126.25	3	2010-11-24 12:24:00	15
10	12358.00	2519.01	3	2010-11-29 10:56:00	10

```
fig = plt.figure(figsize=(10, 10))
ax = fig.add_subplot(projection='3d')  # 111 artinya: 1 baris, 1 kolom, urutan 1

scatter = ax.scatter(non_outliers_df["MonetaryValue"], non_outliers_df["Frequency"], non_outli

ax.set_xlabel('Monetary Value')
ax.set_ylabel('Frequency')
ax.set_zlabel('Recency')

ax.set_title('3D Scatter Plot of Customer Data')

plt.show()
```




```
scaler = StandardScaler()

scaled_data = scaler.fit_transform(non_outliers_df[["MonetaryValue", "Frequency", "Recency"]])

scaled_data
```

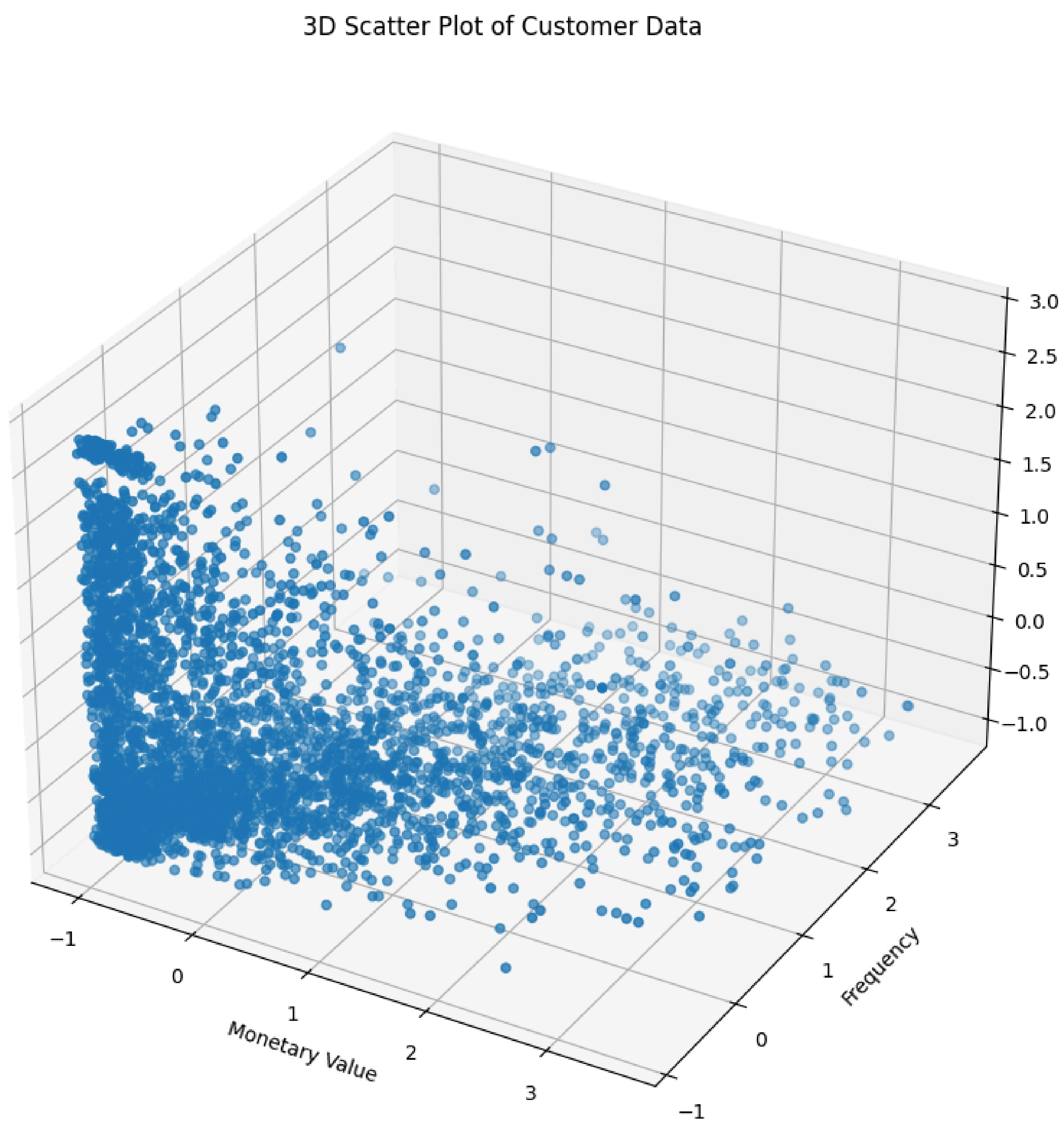
```
array([[-0.87594534, -0.38488934,  0.68214853],
       [ 0.5355144 , -0.38488934, -0.96925093],
       [-0.81258645, -0.83063076, -0.24548944],
       ...,
       [-0.62197163, -0.83063076,  2.01753946],
       [ 0.44146683, -0.38488934,  0.14187587],
       [ 1.72488781,  0.50659348, -0.81634357]], shape=(3809, 3))
```

```
scaled_data_df = pd.DataFrame(scaled_data, index=non_outliers_df.index, columns=("MonetaryValu
scaled_data_df
```

	MonetaryValue	Frequency	Recency
0	-0.88	-0.38	0.68
1	0.54	-0.38	-0.97
2	-0.81	-0.83	-0.25
3	1.63	-0.38	-0.56
4	-0.72	-0.83	-0.89
...	...	...	...
4280	-0.30	1.40	-0.82
4281	-0.58	-0.83	-0.32
4282	-0.62	-0.83	2.02
4283	0.44	-0.38	0.14
4284	1.72	0.51	-0.82

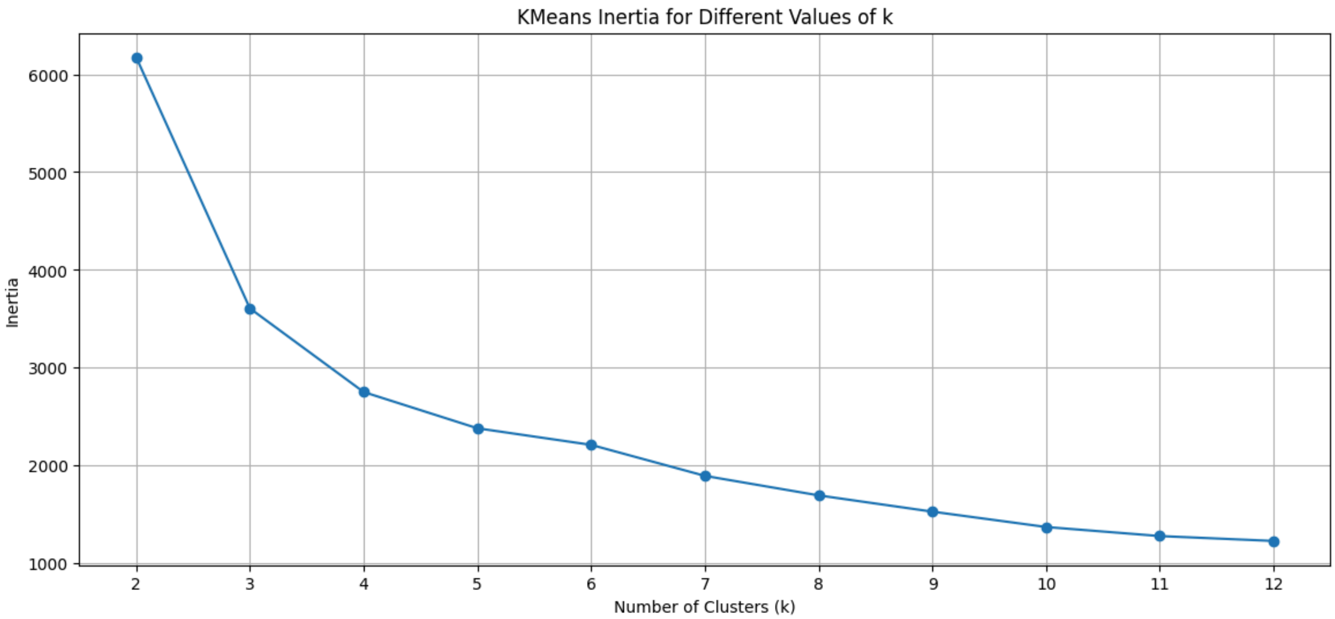
3809 rows × 3 columns

```
fig = plt.figure(figsize=(10, 10))
ax = fig.add_subplot(projection='3d')  # 111 artinya: 1 baris, 1 kolom, urutan 1

scatter = ax.scatter(scaled_data_df["MonetaryValue"], scaled_data_df["Frequency"], scaled_data

ax.set_xlabel('Monetary Value')
ax.set_ylabel('Frequency')
ax.set_zlabel('Recency')

ax.set_title('3D Scatter Plot of Customer Data')

plt.show()
```



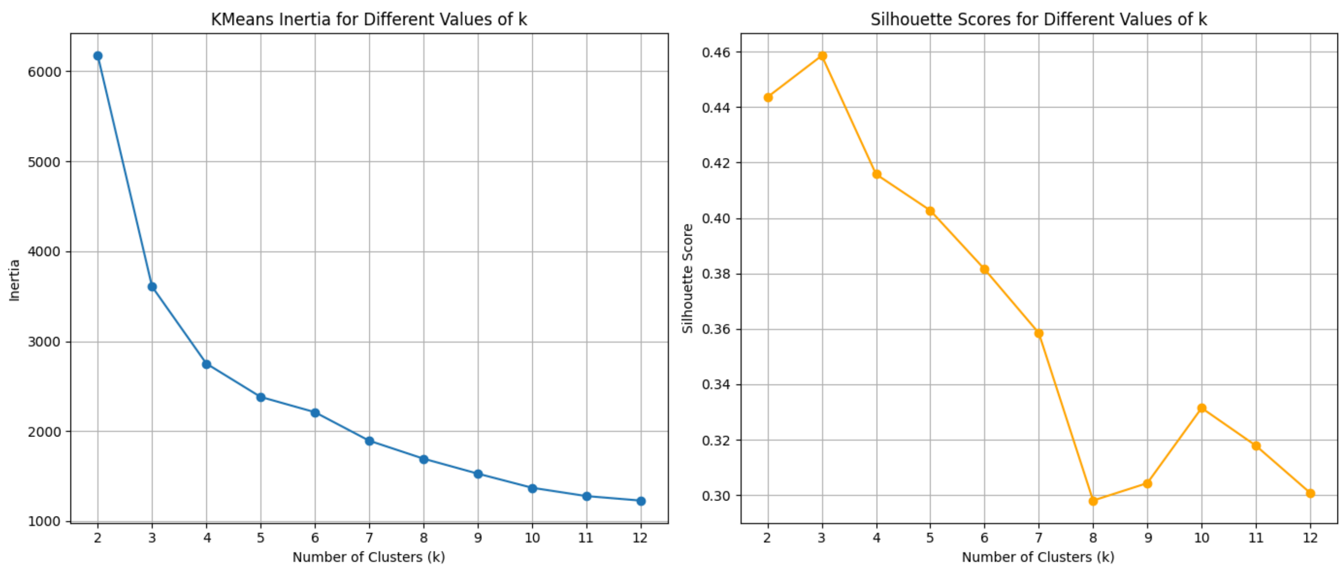

```
max_k = 12

inertia = []
k_values = range(2, max_k + 1)

for k in k_values:

    kmeans = KMeans(n_clusters=k, random_state=42, max_iter=1000)

    kmeans.fit_predict(scaled_data_df)

    inertia.append(kmeans.inertia_)

plt.figure(figsize=(14, 6))
```

```
plt.plot(k_values, inertia, marker='o')
plt.title('KMeans Inertia for Different Values of k')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Inertia')
plt.xticks(k_values)
plt.grid(True)
```




```
max_k = 12

inertia = []
silhouette_scores = []
k_values = range(2, max_k + 1)

for k in k_values:

    kmeans = KMeans(n_clusters=k, random_state=42, max_iter=1000)

    cluester_labels = kmeans.fit_predict(scaled_data_df)

    sil_score = silhouette_score(scaled_data_df, cluester_labels)

    silhouette_scores.append(sil_score)

    inertia.append(kmeans.inertia_)

plt.figure(figsize=(14, 6))

plt.subplot(1, 2, 1)
plt.plot(k_values, inertia, marker='o')
plt.title('KMeans Inertia for Different Values of k')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Inertia')
plt.xticks(k_values)
plt.grid(True)

plt.subplot(1, 2, 2)
```

```
plt.plot(k_values, silhouette_scores, marker='o', color='orange')
plt.title('Silhouette Scores for Different Values of k')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Silhouette Score')
plt.xticks(k_values)
plt.grid(True)

plt.tight_layout()
plt.show()
```




```
kmeans = KMeans(n_clusters=4, random_state=42, max_iter=1000)

cluster_labels = kmeans.fit_predict(scaled_data_df)

cluster_labels
```

```
array([1, 0, 2, ..., 1, 0, 0], shape=(3809,), dtype=int32)
```

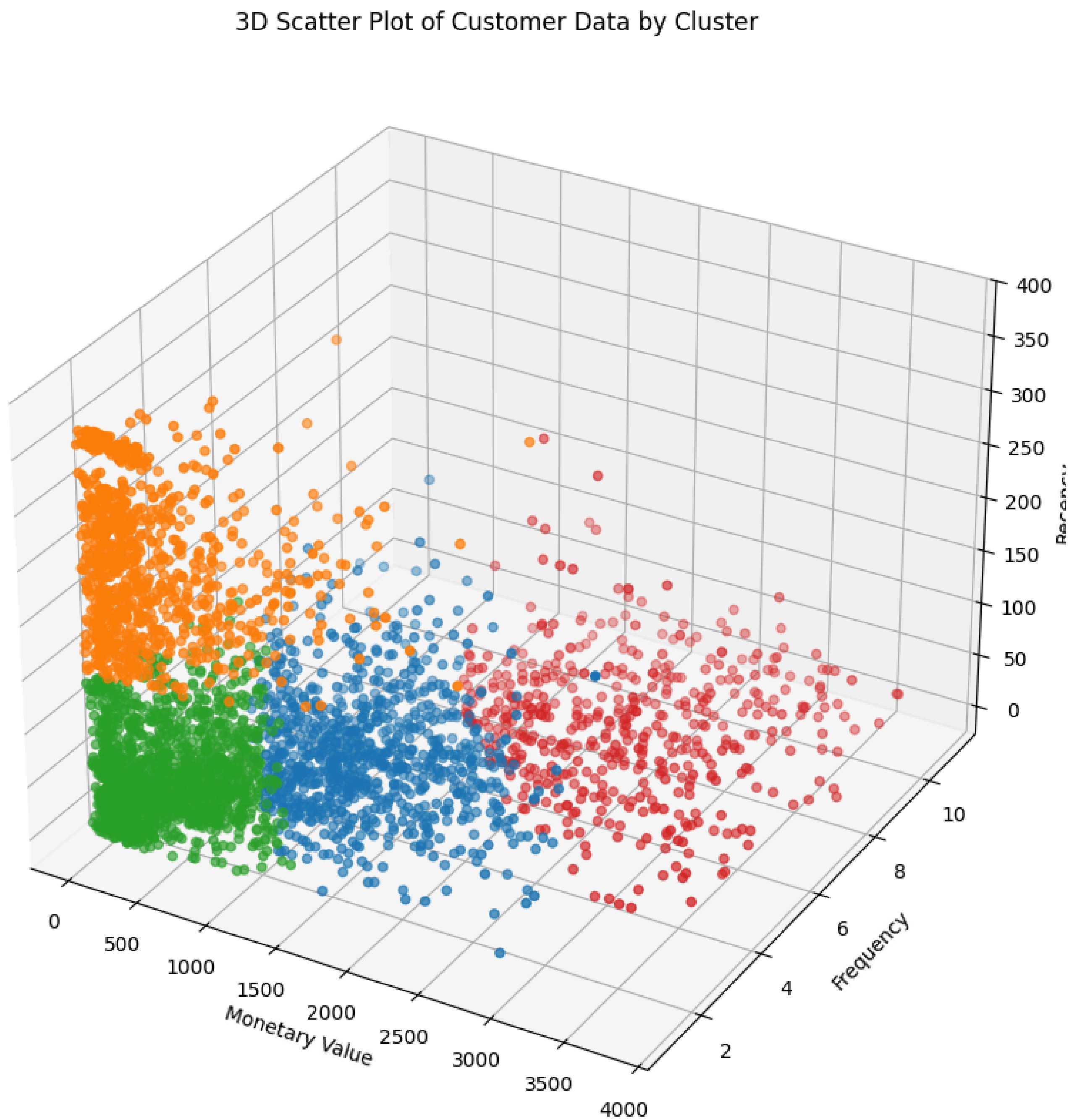
```
non_outliers_df["Cluster"] = cluster_labels

non_outliers_df
```

```
C:\Users\MSI KATANA 15\AppData\Local\Temp\ipykernel_18036\3577770544.py:1: SettingWithCopyWar
ning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guid
e/indexing.html#returning-a-view-versus-a-copy
  non_outliers_df["Cluster"] = cluster_labels
```

	Customer ID	MonetaryValue	Frequency	LastInvoiceDate	Recency	Cluster
0	12346.00	169.36	2	2010-06-28 13:53:00	164	1
1	12347.00	1323.32	2	2010-12-07 14:57:00	2	0
2	12348.00	221.16	1	2010-09-27 14:59:00	73	2
3	12349.00	2221.14	2	2010-10-28 08:23:00	42	0
4	12351.00	300.93	1	2010-11-29 15:23:00	10	2
...	...	...	...	...	...	...
4280	18283.00	641.77	6	2010-11-22 15:30:00	17	0
4281	18284.00	411.68	1	2010-10-04 11:33:00	66	2
4282	18285.00	377.00	1	2010-02-17 10:24:00	295	1
4283	18286.00	1246.43	2	2010-08-20 11:57:00	111	0
4284	18287.00	2295.71	4	2010-11-22 11:51:00	17	0

3809 rows × 6 columns

```python
cluster_colors = {0: '#1f77b4',  # Blue
                  1: '#ff7f0e',  # Orange
                  2: '#2ca02c',  # Green
                  3: '#d62728'}  # Red

colors = non_outliers_df['Cluster'].map(cluster_colors)

fig = plt.figure(figsize=(10, 10))
ax = fig.add_subplot(projection='3d')

scatter = ax.scatter(non_outliers_df['MonetaryValue'],
                     non_outliers_df['Frequency'],
                     non_outliers_df['Recency'],
                     c=colors,  # Use mapped solid colors
                     marker='o')

ax.set_xlabel('Monetary Value')
ax.set_ylabel('Frequency')
ax.set_zlabel('Recency')

ax.set_title('3D Scatter Plot of Customer Data by Cluster')

plt.show()
```

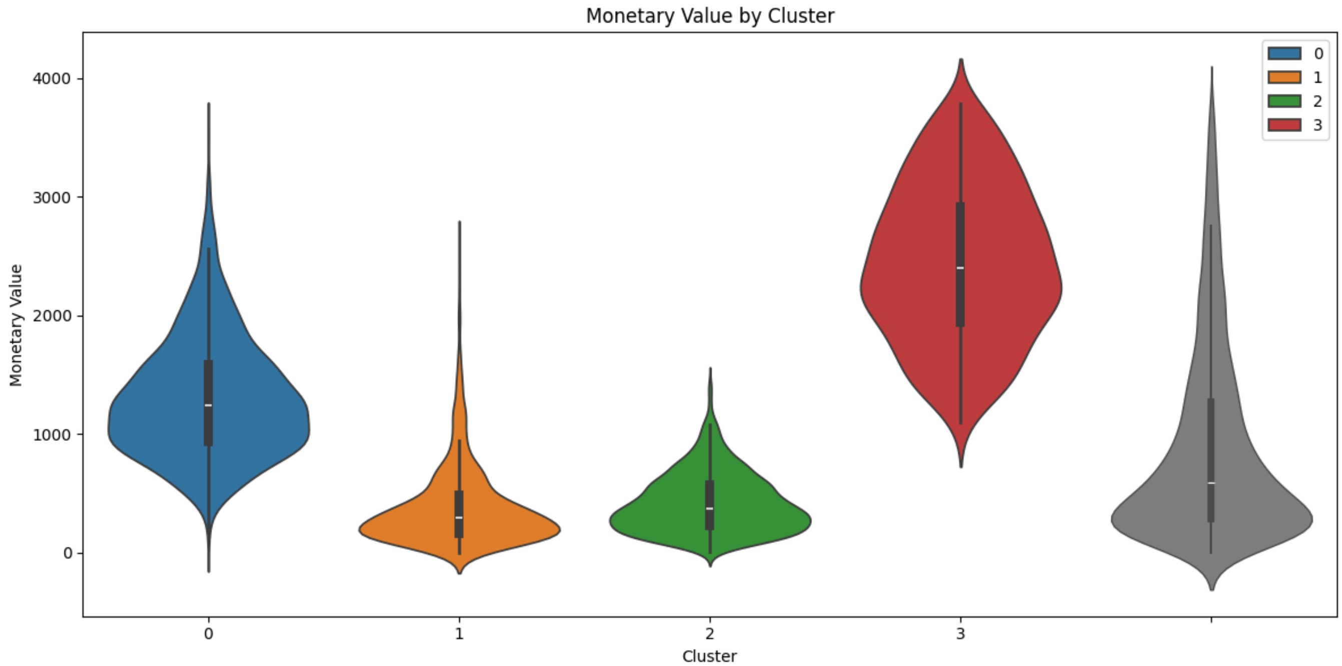


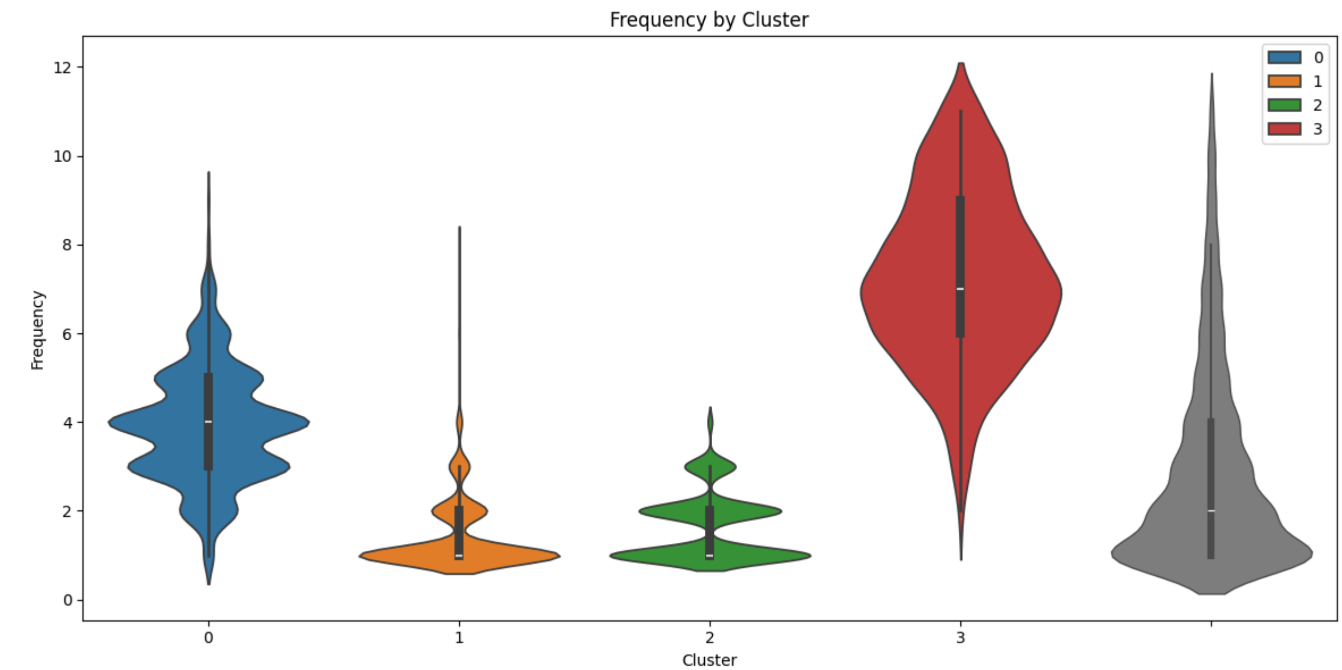

```
# Hitung rata-rata tiap metrik RFM berdasarkan Cluster
cluster_summary = non_outliers_df.groupby('Cluster')[['MonetaryValue', 'Frequency', 'Recency']

# Hitung juga jumlah orang di setiap cluster (biar tau mana yang mayoritas)
cluster_counts = non_outliers_df['Cluster'].value_counts().reset_index()
cluster_counts.columns = ['Cluster', 'Count']

# Gabungkan keduanya
final_summary = cluster_summary.merge(cluster_counts, on='Cluster')

# Tampilkan dengan format angka 2 desimal biar rapi
print("=== RATA-RATA PROFIL PER KLASTER ===")
display(final_summary.round(2))
```

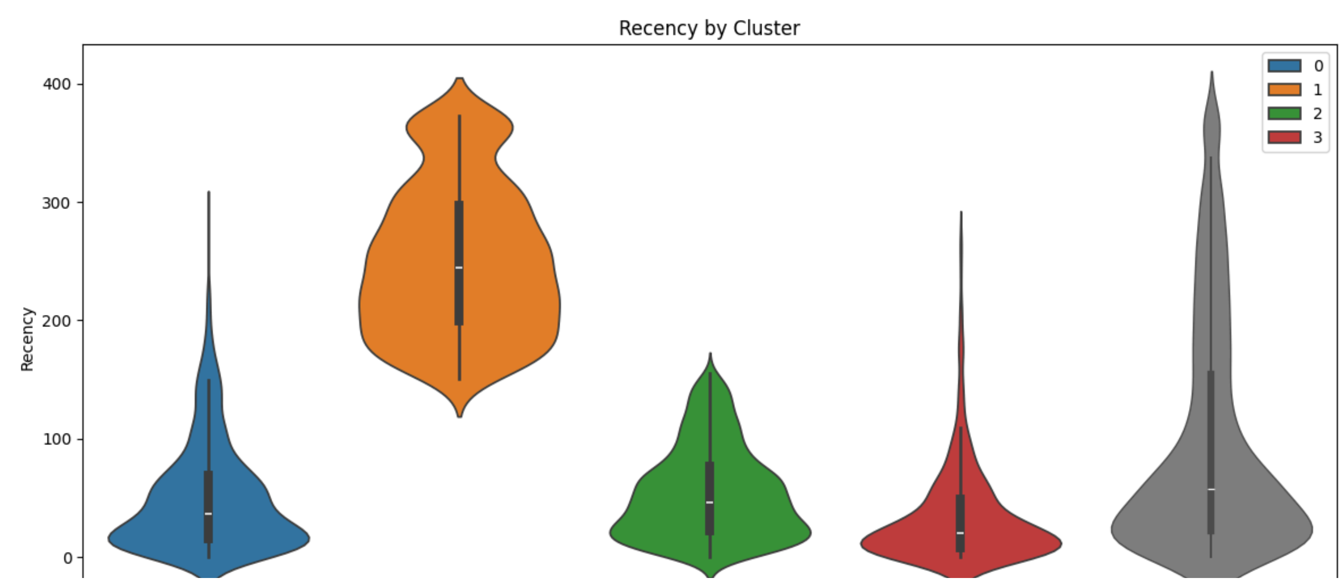
```
=== RATA-RATA PROFIL PER KLASTER ===
```

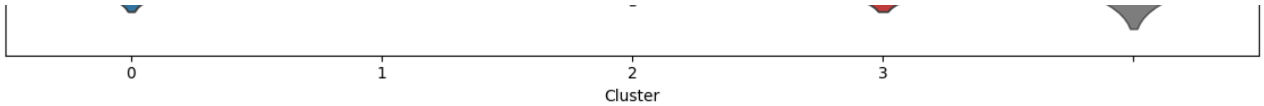
	Cluster	MonetaryValue	Frequency	Recency	Count
0	0	1308.62	3.91	49.73	914
1	1	384.54	1.43	251.17	902
2	2	417.95	1.64	54.07	1499
3	3	2436.09	7.24	33.89	494

```
plt.figure(figsize=(12, 18))

plt.subplot(3, 1, 1)
sns.violinplot(x=non_outliers_df['Cluster'], y=non_outliers_df['MonetaryValue'], palette=clust
sns.violinplot(y=non_outliers_df['MonetaryValue'], color='gray', linewidth=1.0)
plt.title('Monetary Value by Cluster')
plt.ylabel('Monetary Value')

plt.subplot(3, 1, 2)
sns.violinplot(x=non_outliers_df['Cluster'], y=non_outliers_df['Frequency'], palette=cluster_c
sns.violinplot(y=non_outliers_df['Frequency'], color='gray', linewidth=1.0)
plt.title('Frequency by Cluster')
plt.ylabel('Frequency')


plt.subplot(3, 1, 3)
sns.violinplot(x=non_outliers_df['Cluster'], y=non_outliers_df['Recency'], palette=cluster_col
sns.violinplot(y=non_outliers_df['Recency'], color='gray', linewidth=1.0)
plt.title('Recency by Cluster')
plt.ylabel('Recency')

plt.tight_layout()
plt.show()
```

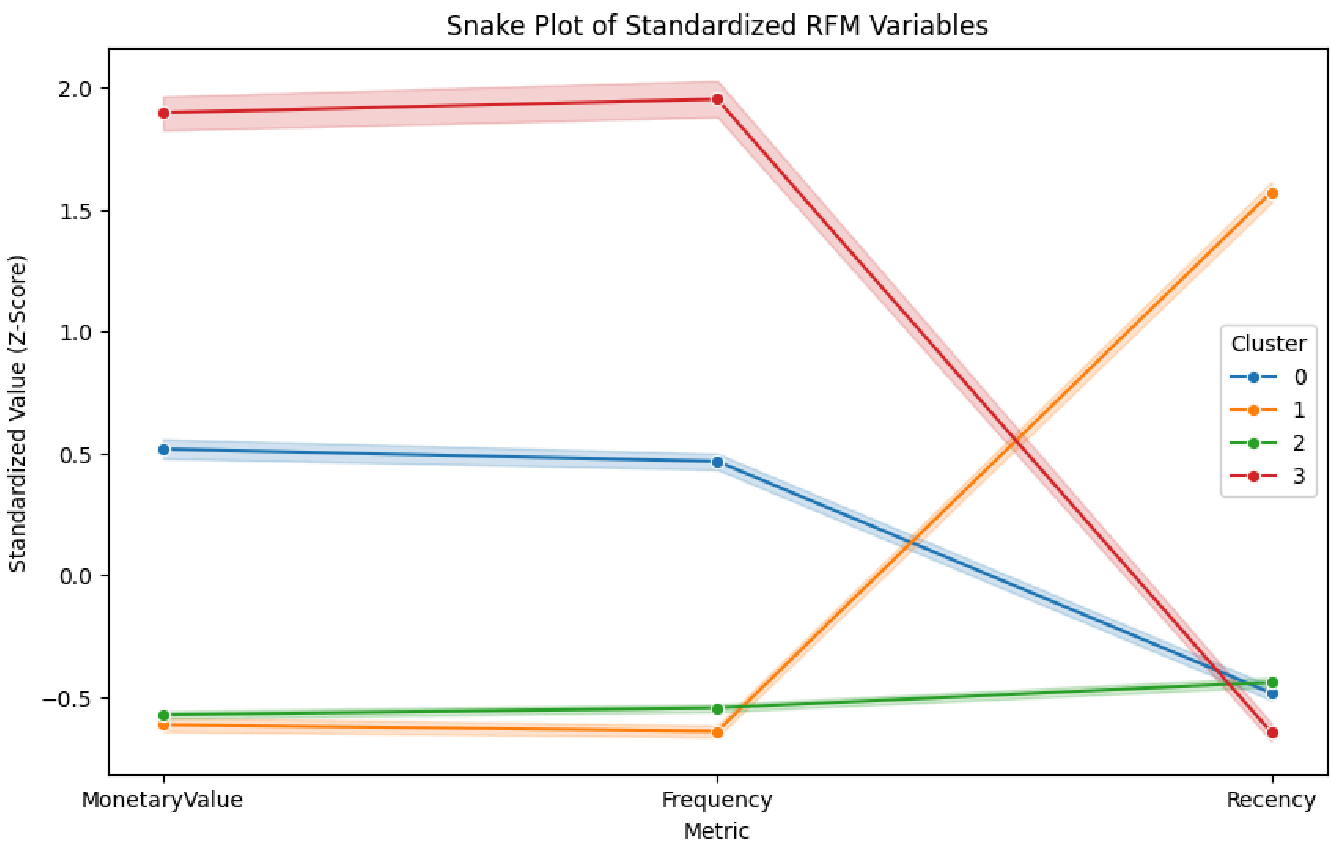

Monetary Value by Cluster
Monetary Value
Cluster
0
1
2
3
0
1000
2000
3000
4000


Frequency by Cluster
Frequency
Cluster
0
1
2
3
0
2
4
6
8
10
12


Recency by Cluster
Recency
0
1
2
3
0
100
200
300
400




```
# --- Kodingan Visualisasi Snake Plot ---
snake_data = scaled_data_df.copy()
snake_data['Cluster'] = cluster_labels

# Reset index biar indexnya jadi kolom biasa, TAPI kita gak usah panggil namanya spesifik
snake_melt = pd.melt(snake_data.reset_index(),
                     id_vars=['Cluster'], # Cukup Cluster aja sebagai identitas
                     value_vars=['MonetaryValue', 'Frequency', 'Recency'],
                     var_name='Metric', value_name='Value')

plt.figure(figsize=(10, 6))
sns.lineplot(data=snake_melt, x='Metric', y='Value', hue='Cluster', palette=cluster_colors, ma
plt.title('Snake Plot of Standardized RFM Variables')
plt.xlabel('Metric')
plt.ylabel('Standardized Value (Z-Score)')
plt.legend(title='Cluster')
plt.show()
```




```
overlap_indices = monetary_outliers_df.index.intersection(frequency_outliers_df.index)

monetary_only_outliers = monetary_outliers_df.drop(overlap_indices)
frequency_only_outliers = frequency_outliers_df.drop(overlap_indices)
monetary_and_frequency_outliers = monetary_outliers_df.loc[overlap_indices]

monetary_only_outliers["Cluster"] = -1
```

```
frequency_only_outliers["Cluster"] = -2
monetary_and_frequency_outliers["Cluster"] = -3

outlier_clusters_df = pd.concat([monetary_only_outliers, frequency_only_outliers, monetary_and

outlier_clusters_df
```

	Customer ID	MonetaryValue	Frequency	LastInvoiceDate	Recency	Cluster
9	12357.00	11229.99	1	2010-11-16 10:05:00	23	-1
25	12380.00	4782.84	4	2010-08-31 14:54:00	100	-1
42	12409.00	12346.62	4	2010-10-15 10:24:00	55	-1
48	12415.00	19468.84	4	2010-11-29 15:07:00	10	-1
61	12431.00	4145.52	11	2010-12-01 10:03:00	8	-1
...	...	...	...	...	...	...
4235	18223.00	7516.31	12	2010-11-17 12:20:00	22	-3
4236	18225.00	7545.14	15	2010-12-09 15:46:00	0	-3
4237	18226.00	6650.83	15	2010-11-26 15:51:00	13	-3
4241	18231.00	4791.80	23	2010-10-29 14:17:00	41	-3
4262	18260.00	7318.91	17	2010-11-30 12:25:00	9	-3

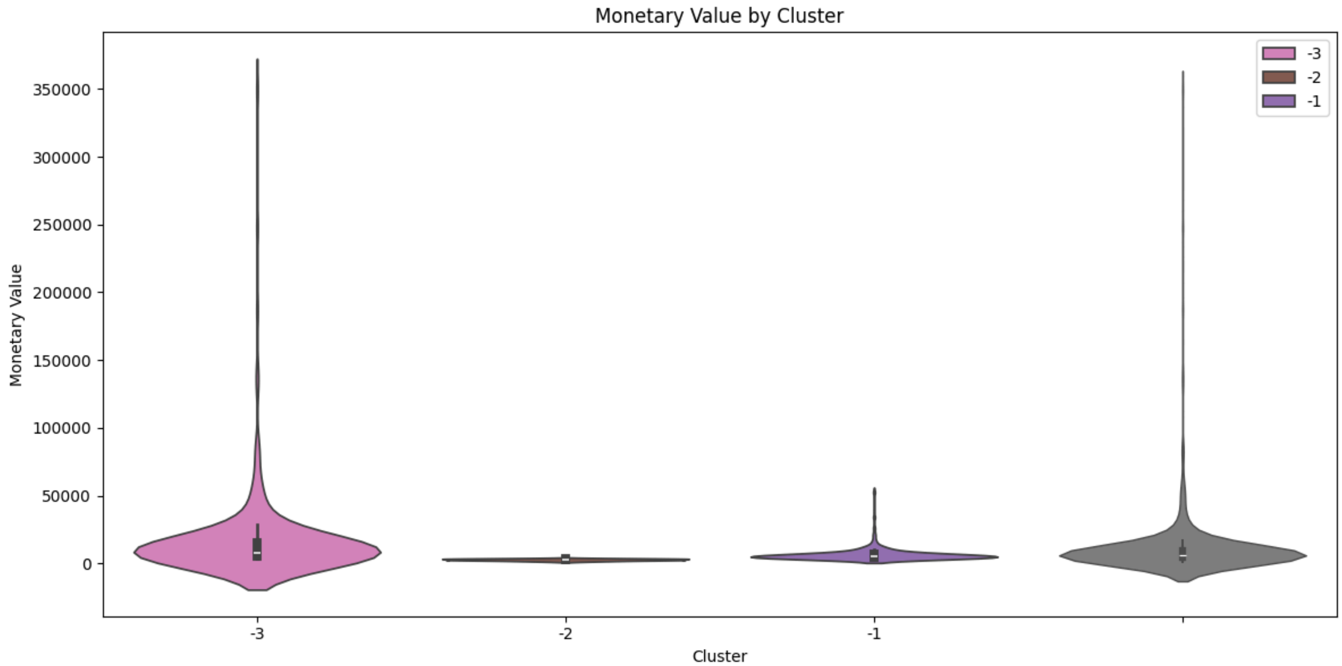
476 rows × 6 columns

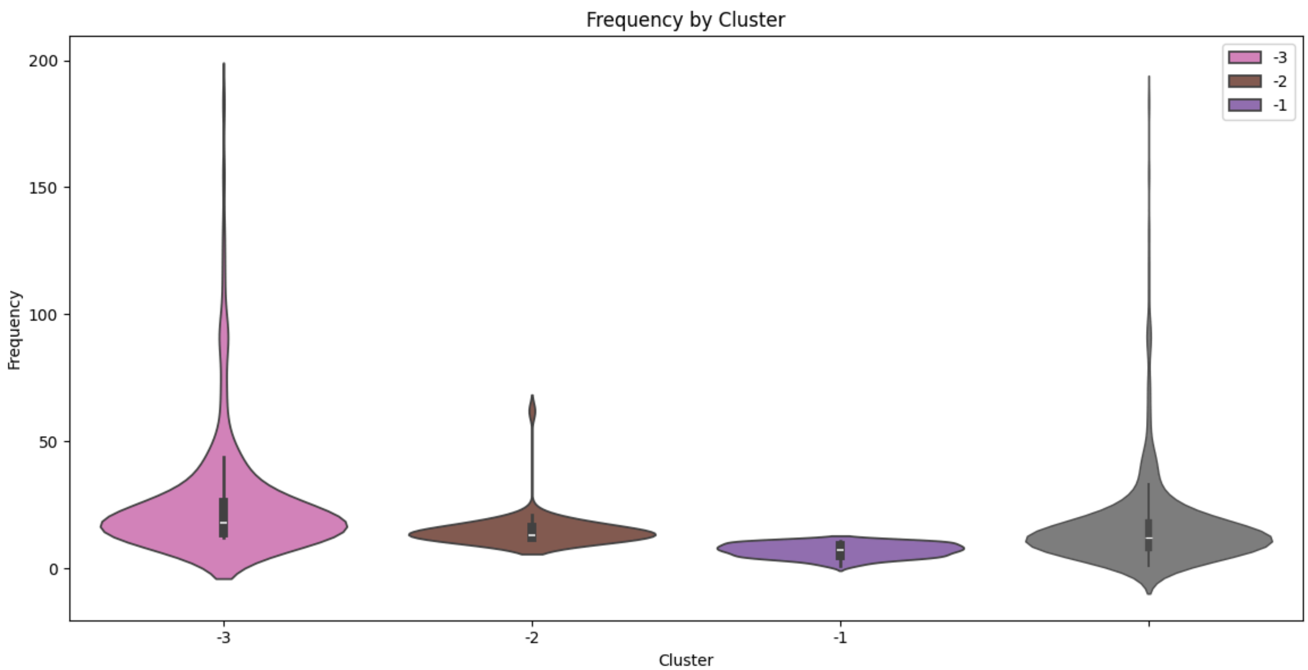
```
# Hitung rata-rata tiap metrik RFM berdasarkan Cluster
cluster_summary = outlier_clusters_df.groupby('Cluster')[['MonetaryValue', 'Frequency', 'Recer

# Hitung juga jumlah orang di setiap cluster (biar tau mana yang mayoritas)
cluster_counts = outlier_clusters_df['Cluster'].value_counts().reset_index()
cluster_counts.columns = ['Cluster', 'Count']

# Gabungkan keduanya
final_summary = cluster_summary.merge(cluster_counts, on='Cluster')

# Tampilkan dengan format angka 2 desimal biar rapi
print("=== RATA-RATA PROFIL PER KLASTER ===")
display(final_summary.round(2))
```

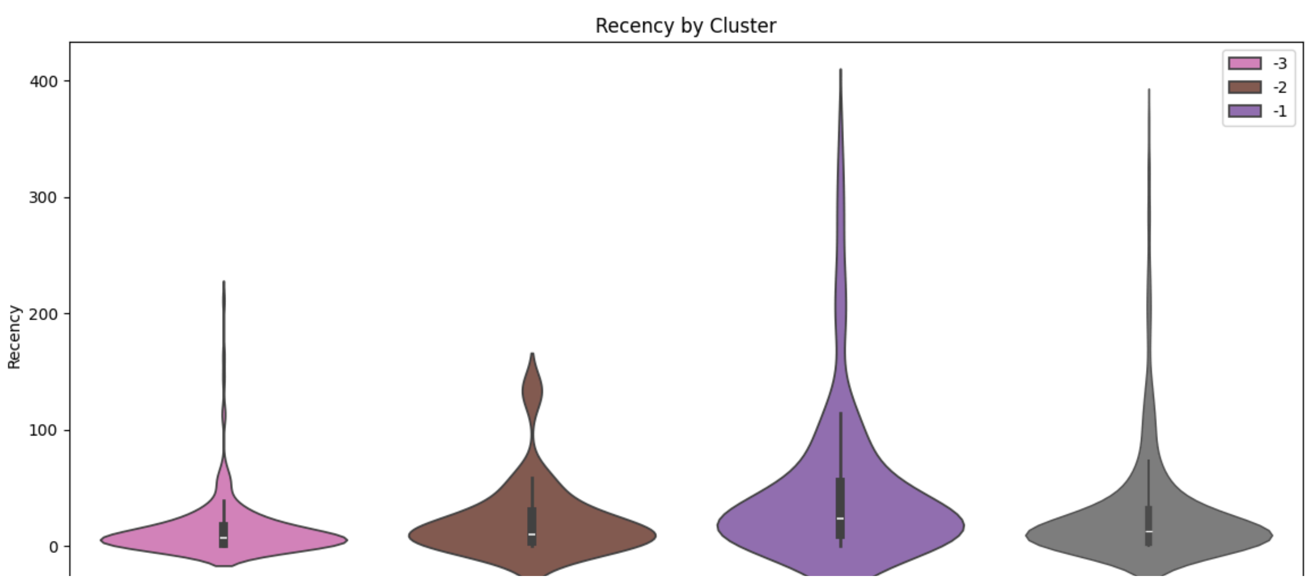
```
=== RATA-RATA PROFIL PER KLASTER ===
```

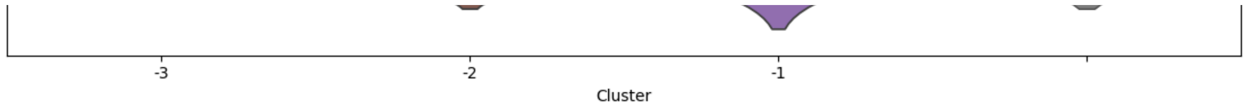
	Cluster	MonetaryValue	Frequency	Recency	Count
0	-3	17147.66	25.87	14.45	226
1	-2	2734.69	15.04	23.08	53
2	-1	6498.45	7.19	47.91	197

```
cluster_colors = {-1: '#9467bd',
                  -2: '#8c564b',
                  -3: '#e377c2'}

plt.figure(figsize=(12, 18))

plt.subplot(3, 1, 1)
sns.violinplot(x=outlier_clusters_df['Cluster'], y=outlier_clusters_df['MonetaryValue'], palet
sns.violinplot(y=outlier_clusters_df['MonetaryValue'], color='gray', linewidth=1.0)
plt.title('Monetary Value by Cluster')
plt.ylabel('Monetary Value')

plt.subplot(3, 1, 2)
sns.violinplot(x=outlier_clusters_df['Cluster'], y=outlier_clusters_df['Frequency'], palette=c
sns.violinplot(y=outlier_clusters_df['Frequency'], color='gray', linewidth=1.0)
plt.title('Frequency by Cluster')
plt.ylabel('Frequency')

plt.subplot(3, 1, 3)
sns.violinplot(x=outlier_clusters_df['Cluster'], y=outlier_clusters_df['Recency'], palette=clu
sns.violinplot(y=outlier_clusters_df['Recency'], color='gray', linewidth=1.0)
plt.title('Recency by Cluster')
plt.ylabel('Recency')

plt.tight_layout()
plt.show()
```

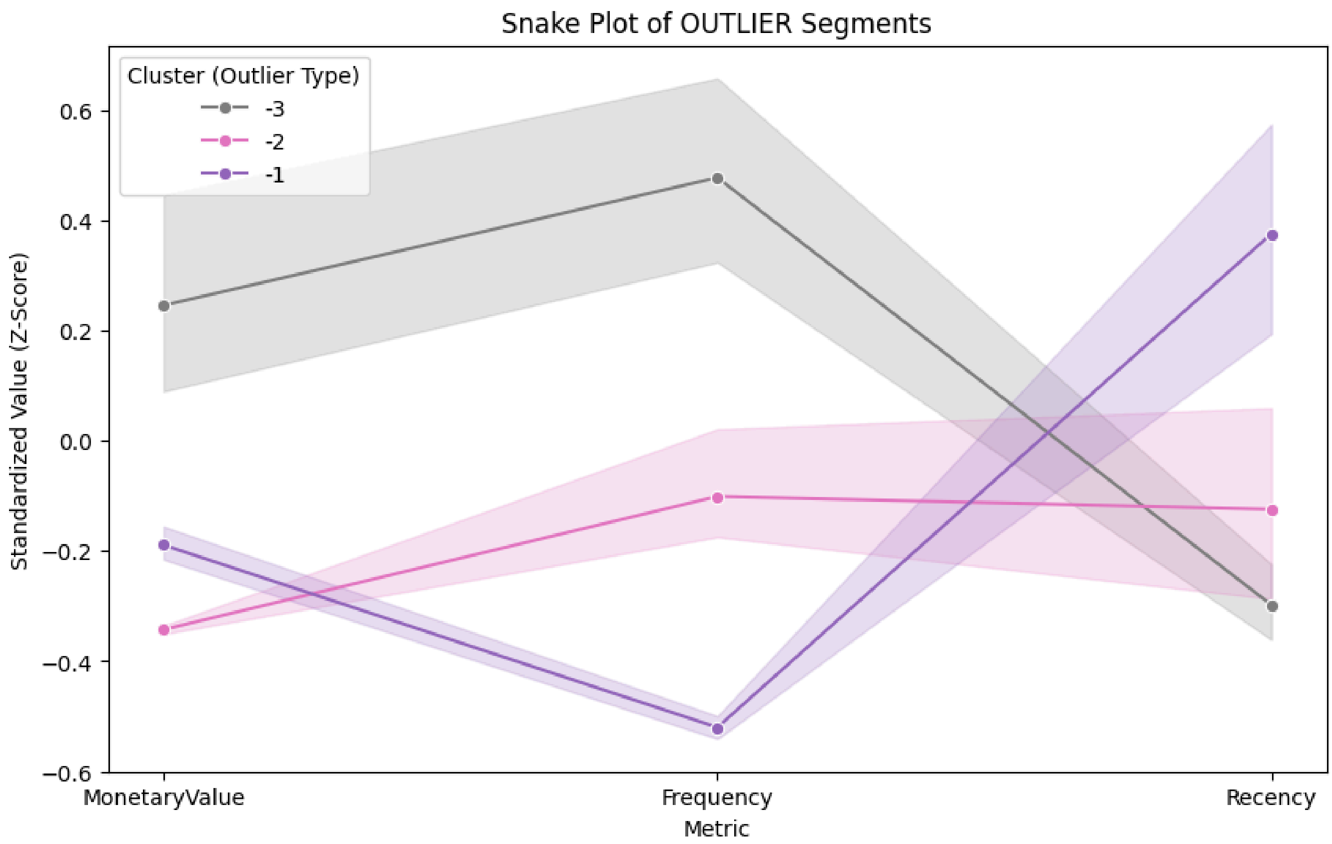

Monetary Value by Cluster
Monetary Value
350000
300000
250000
200000
150000
100000
50000
0
-3
-2
-1
Cluster
-3
-2
-1


Frequency by Cluster
Frequency
200
150
100
50
0
-3
-2
-1
Cluster
-3
-2
-1


Recency by Cluster
Recency
400
300
200
100
0
-3
-2
-1




```
# 1. Scaling Data Outlier
scaler_outlier = StandardScaler()
scaled_outliers = scaler_outlier.fit_transform(outlier_clusters_df[["MonetaryValue", "Frequenc

# 2. Bikin DataFrame hasil scaling
scaled_outliers_df = pd.DataFrame(scaled_outliers,
                                  index=outlier_clusters_df.index,
                                  columns=["MonetaryValue", "Frequency", "Recency"])

# 3. Tempel Label Clusternya (-1, -2, -3)
scaled_outliers_df['Cluster'] = outlier_clusters_df['Cluster']

# 4. Melt Data (Bentuk Ular)
snake_melt_outlier = pd.melt(scaled_outliers_df.reset_index(),
                             id_vars=['Cluster'],
                             value_vars=['MonetaryValue', 'Frequency', 'Recency'],
                             var_name='Metric', value_name='Value')

# 5. Gambar!
# Kita pakai warna khusus biar beda (Ungu, Pink, Abu)
outlier_palette = {-1: '#9467bd', -2: '#e377c2', -3: '#7f7f7f'}

plt.figure(figsize=(10, 6))
sns.lineplot(data=snake_melt_outlier, x='Metric', y='Value', hue='Cluster', palette=outlier_pa
plt.title('Snake Plot of OUTLIER Segments')
plt.xlabel('Metric')
plt.ylabel('Standardized Value (Z-Score)')
plt.legend(title='Cluster (Outlier Type)')
plt.show()
```




```
cluster_labels = {
    0: "RETAIN",
    1: "RE-ENGAGE",
    2: "NURTURE",
    3: "REWARD",
    -1: "PAMPER", # (Manjakan, karena belanja mahal tapi jarang).
    -2: "UPSELL", # (Tawarkan barang lebih mahal, karena sering beli tapi murah).
    -3: "DELIGHT" # (Sangat Bahagiakan, karena ini VVIP Absolute).
}
```

```
full_clustering_df = pd.concat([non_outliers_df, outlier_clusters_df])
# pd.concat: Menggabungkan tabel non_outliers (mayoritas) dengan tabel outliers (elit) menjadi

full_clustering_df
```

	Customer ID	MonetaryValue	Frequency	LastInvoiceDate	Recency	Cluster
0	12346.00	169.36	2	2010-06-28 13:53:00	164	1
1	12347.00	1323.32	2	2010-12-07 14:57:00	2	0
2	12348.00	221.16	1	2010-09-27 14:59:00	73	2
3	12349.00	2221.14	2	2010-10-28 08:23:00	42	0
4	12351.00	300.93	1	2010-11-29 15:23:00	10	2
...	...	...	...	...	...	...
4235	18223.00	7516.31	12	2010-11-17 12:20:00	22	-3
4236	18225.00	7545.14	15	2010-12-09 15:46:00	0	-3
4237	18226.00	6650.83	15	2010-11-26 15:51:00	13	-3
4241	18231.00	4791.80	23	2010-10-29 14:17:00	41	-3
4262	18260.00	7318.91	17	2010-11-30 12:25:00	9	-3

4285 rows × 6 columns

```
full_clustering_df["ClusterLabel"] = full_clustering_df["Cluster"].map(cluster_labels)

full_clustering_df
```

	Customer ID	MonetaryValue	Frequency	LastInvoiceDate	Recency	Cluster	ClusterLabel
0	12346.00	169.36	2	2010-06-28 13:53:00	164	1	RE-ENGAGE
1	12347.00	1323.32	2	2010-12-07 14:57:00	2	0	RETAIN
2	12348.00	221.16	1	2010-09-27 14:59:00	73	2	NURTURE
3	12349.00	2221.14	2	2010-10-28 08:23:00	42	0	RETAIN
4	12351.00	300.93	1	2010-11-29 15:23:00	10	2	NURTURE
...	...	...	...	...	...	...	...
4235	18223.00	7516.31	12	2010-11-17 12:20:00	22	-3	DELIGHT
4236	18225.00	7545.14	15	2010-12-09 15:46:00	0	-3	DELIGHT
4237	18226.00	6650.83	15	2010-11-26 15:51:00	13	-3	DELIGHT
4241	18231.00	4791.80	23	2010-10-29 14:17:00	41	-3	DELIGHT
4262	18260.00	7318.91	17	2010-11-30 12:25:00	9	-3	DELIGHT

4285 rows × 7 columns

```
cluster_counts = full_clustering_df['ClusterLabel'].value_counts()
full_clustering_df["MonetaryValue per 100 pounds"] = full_clustering_df["MonetaryValue"] / 10(
```

```
feature_means = full_clustering_df.groupby('ClusterLabel')[['Recency', 'Frequency', 'Monetary\

fig, ax1 = plt.subplots(figsize=(12, 8))

sns.barplot(x=cluster_counts.index, y=cluster_counts.values, ax=ax1, palette='viridis', hue=cl
ax1.set_ylabel('Number of Customers', color='b')
ax1.set_title('Cluster Distribution with Average Feature Values')

ax2 = ax1.twinx()

sns.lineplot(data=feature_means, ax=ax2, palette='Set2', marker='o')
ax2.set_ylabel('Average Value', color='g')

plt.show()
```